

(12) **United States Patent**
Smith et al.

(10) **Patent No.:** **US 12,399,956 B2**
(45) **Date of Patent:** **Aug. 26, 2025**

(54) **APPARATUS AND METHOD FOR DIRECTED
PROCESS GENERATION**

(71) Applicant: **Strategic Coach**, Toronto (CA)
(72) Inventors: **Barbara Sue Smith**, Toronto (CA);
Daniel J. Sullivan, Toronto (CA)
(73) Assignee: **The Strategic Coach Inc.**, Toronto
(CA)
(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/527,598**
(22) Filed: **Dec. 4, 2023**

(65) **Prior Publication Data**
US 2024/0370528 A1 Nov. 7, 2024

Related U.S. Application Data
(63) Continuation of application No. 18/142,656, filed on
May 3, 2023, now Pat. No. 12,008,080.
(51) **Int. Cl.**
G06F 18/2415 (2023.01)
G06N 20/00 (2019.01)
(52) **U.S. Cl.**
CPC **G06F 18/2415** (2023.01); **G06N 20/00**
(2019.01)

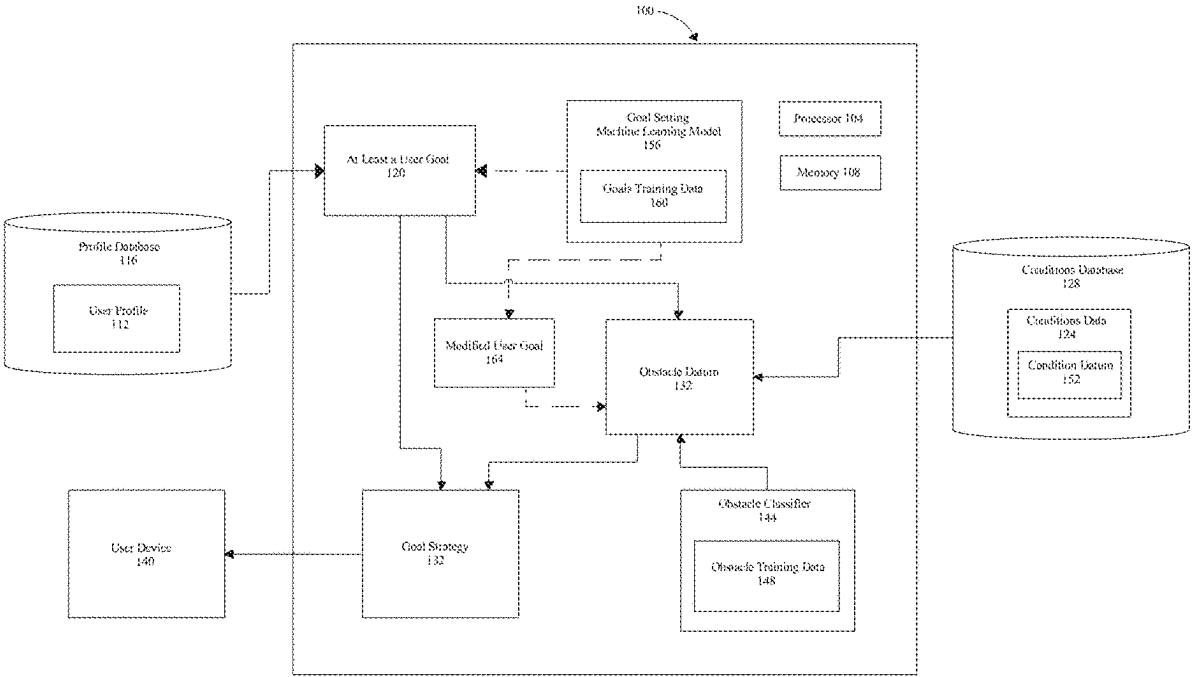
(58) **Field of Classification Search**
CPC G06F 18/2415; G06N 20/00
See application file for complete search history.

(56) **References Cited**
U.S. PATENT DOCUMENTS
7,774,220 B2 8/2010 Sullivan
9,691,046 B1 6/2017 Adler
11,599,592 B1 * 3/2023 Adams G06F 16/906
2013/0282638 A1 10/2013 Baum
2016/0074707 A1 3/2016 Thorpe
2021/0366045 A1 * 11/2021 Young G06Q 50/01

* cited by examiner
Primary Examiner — Hau H Hoang
(74) *Attorney, Agent, or Firm* — Caldwell Intellectual
Property Law

(57) **ABSTRACT**
An apparatus and method. The apparatus including a least a
processor configured to: receive a user profile from a profile
database, identify a plurality of tasks using the user profile,
determine at least an assignable task, receive internal person-
nel assignment data from a personnel database, deter-
mine internal personnel additional tasks, receive posting
data, determine external personnel assignment data as a
function of the posting data, generate a personnel list as a
function of the internal personnel assignment data and the
external personnel assignment data, generate at least one
personnel assignment for the assignable task as a function of
the personnel list; and transmit the at least a personnel
assignment to a user device.

20 Claims, 9 Drawing Sheets



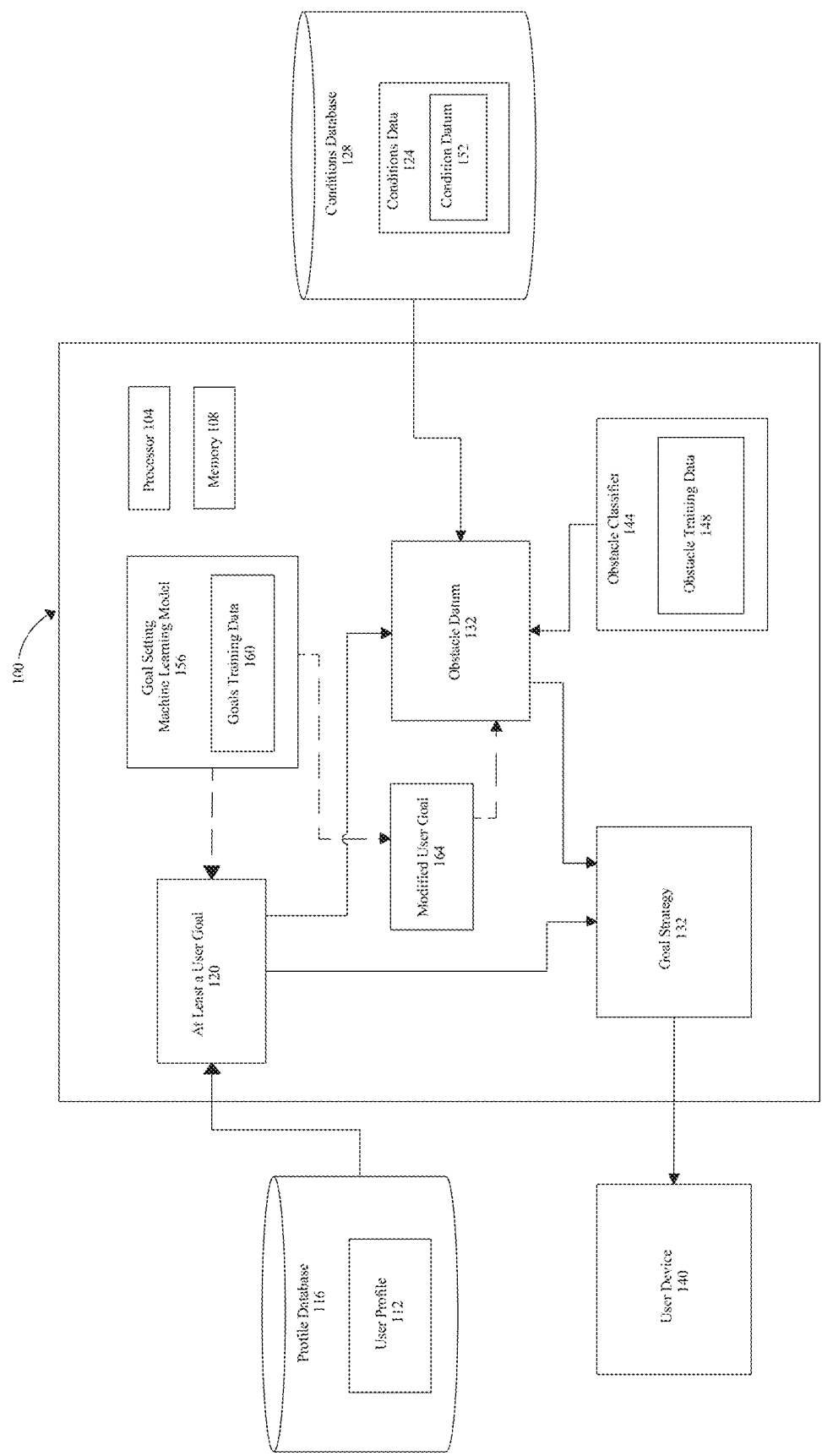


FIG. 1

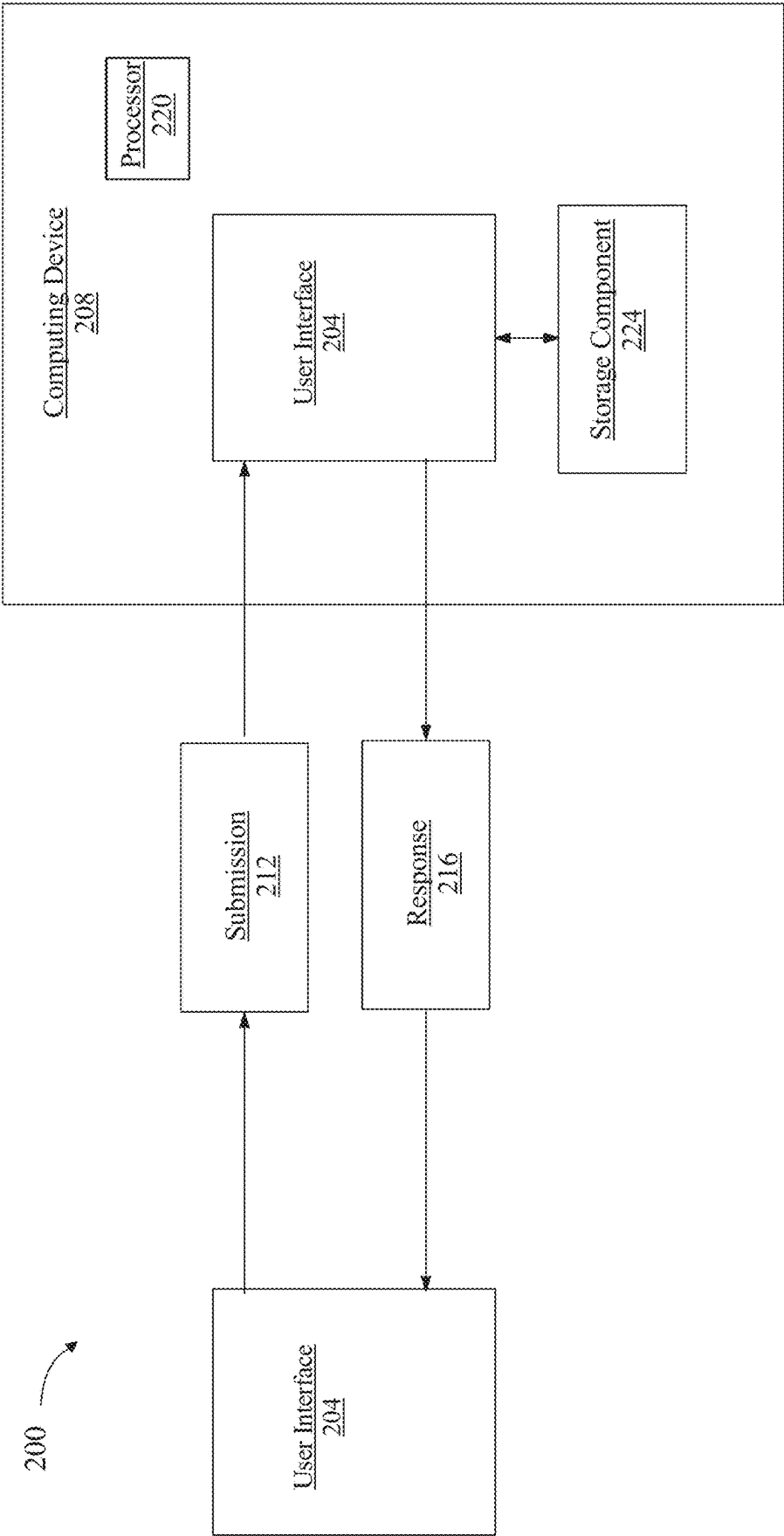


FIG. 2

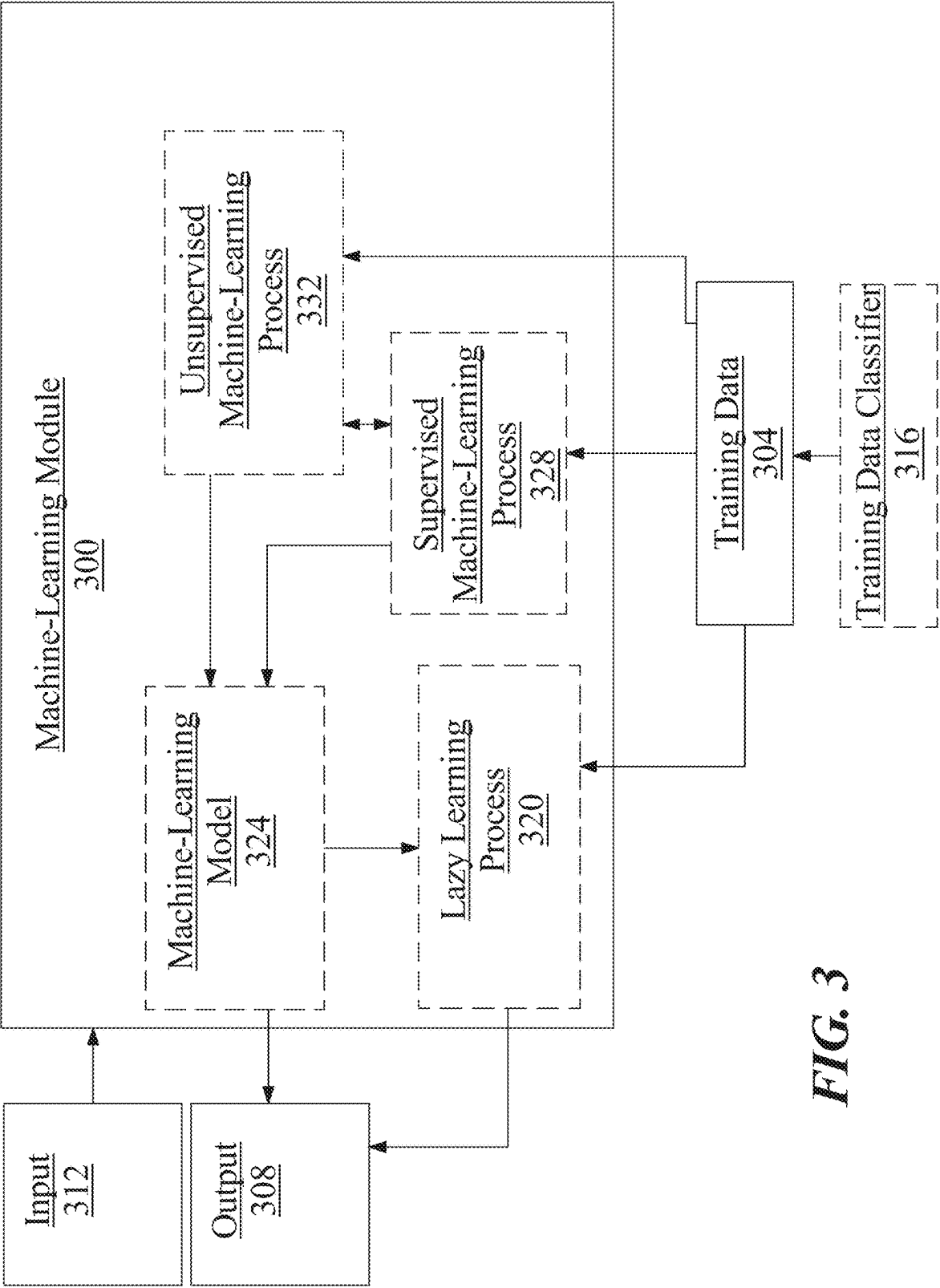


FIG. 3

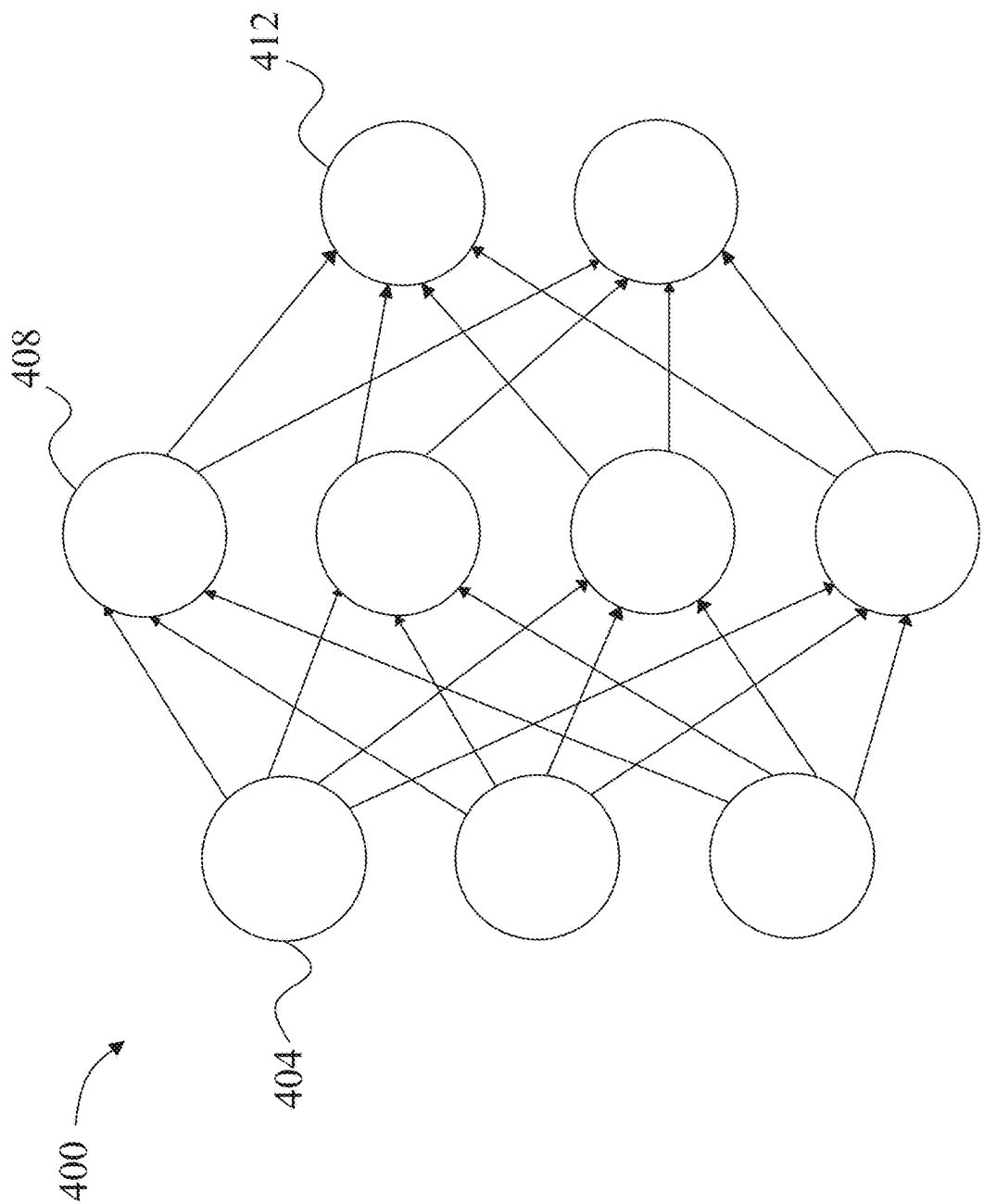


FIG.4

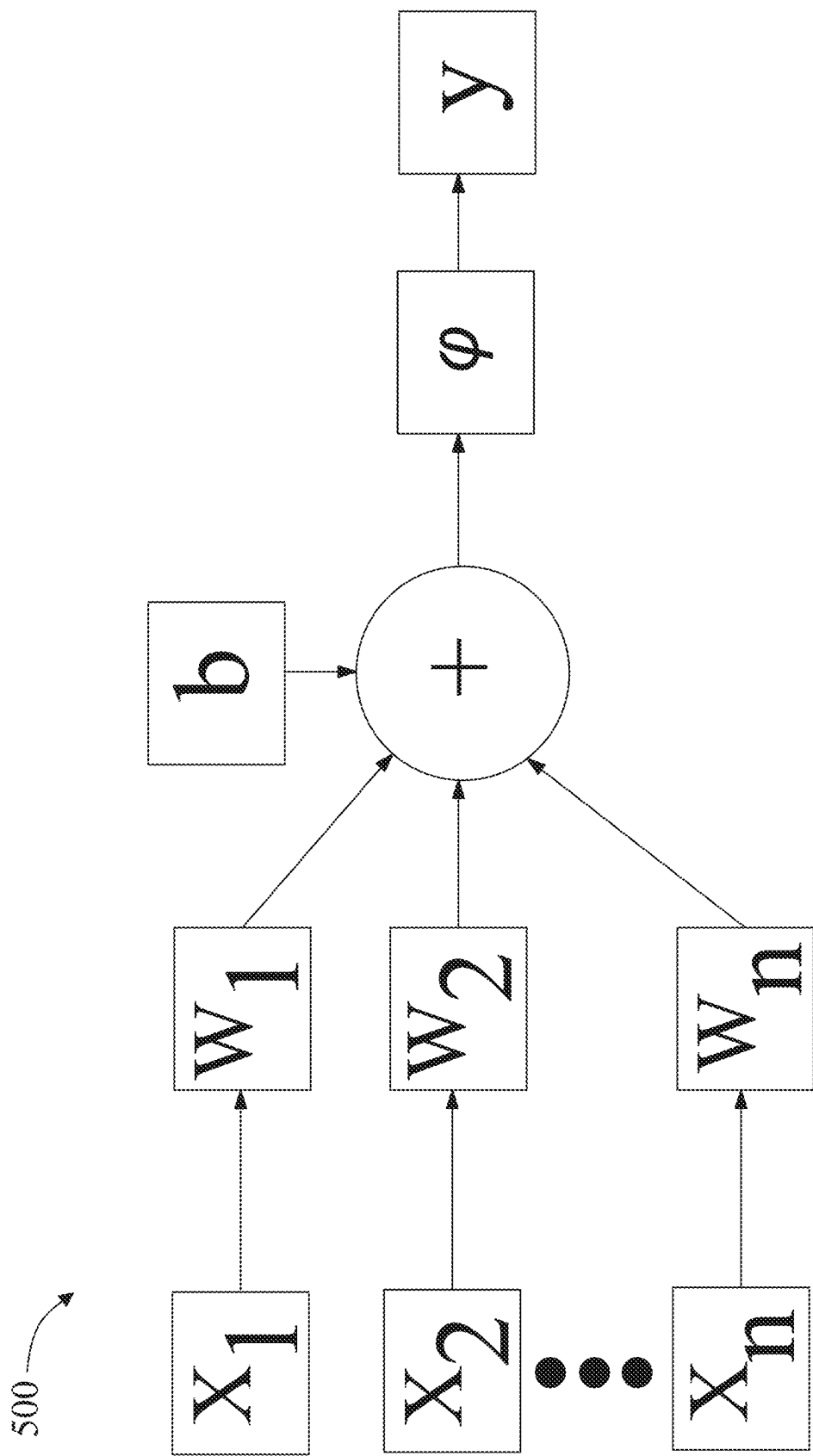


FIG. 5

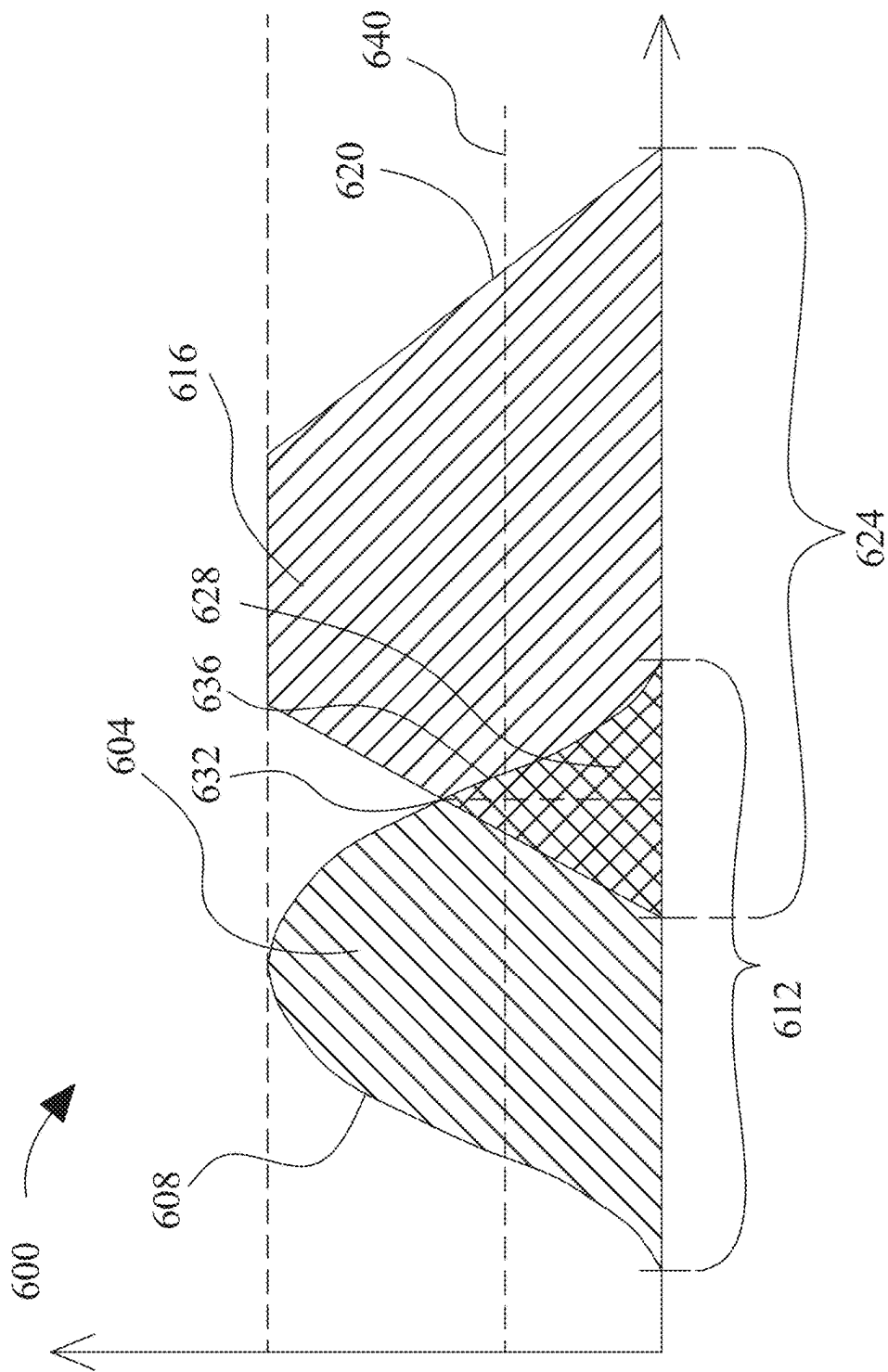
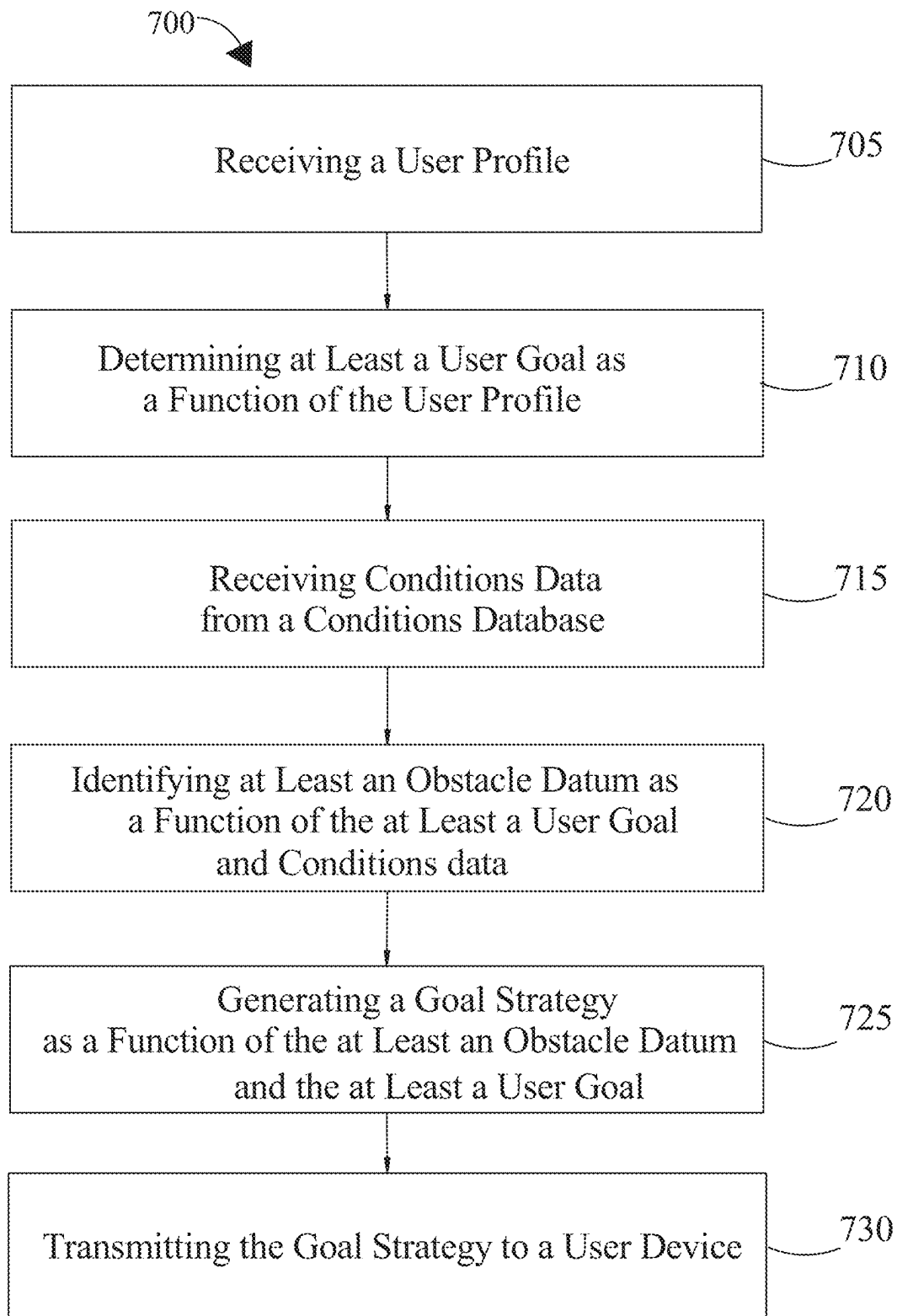
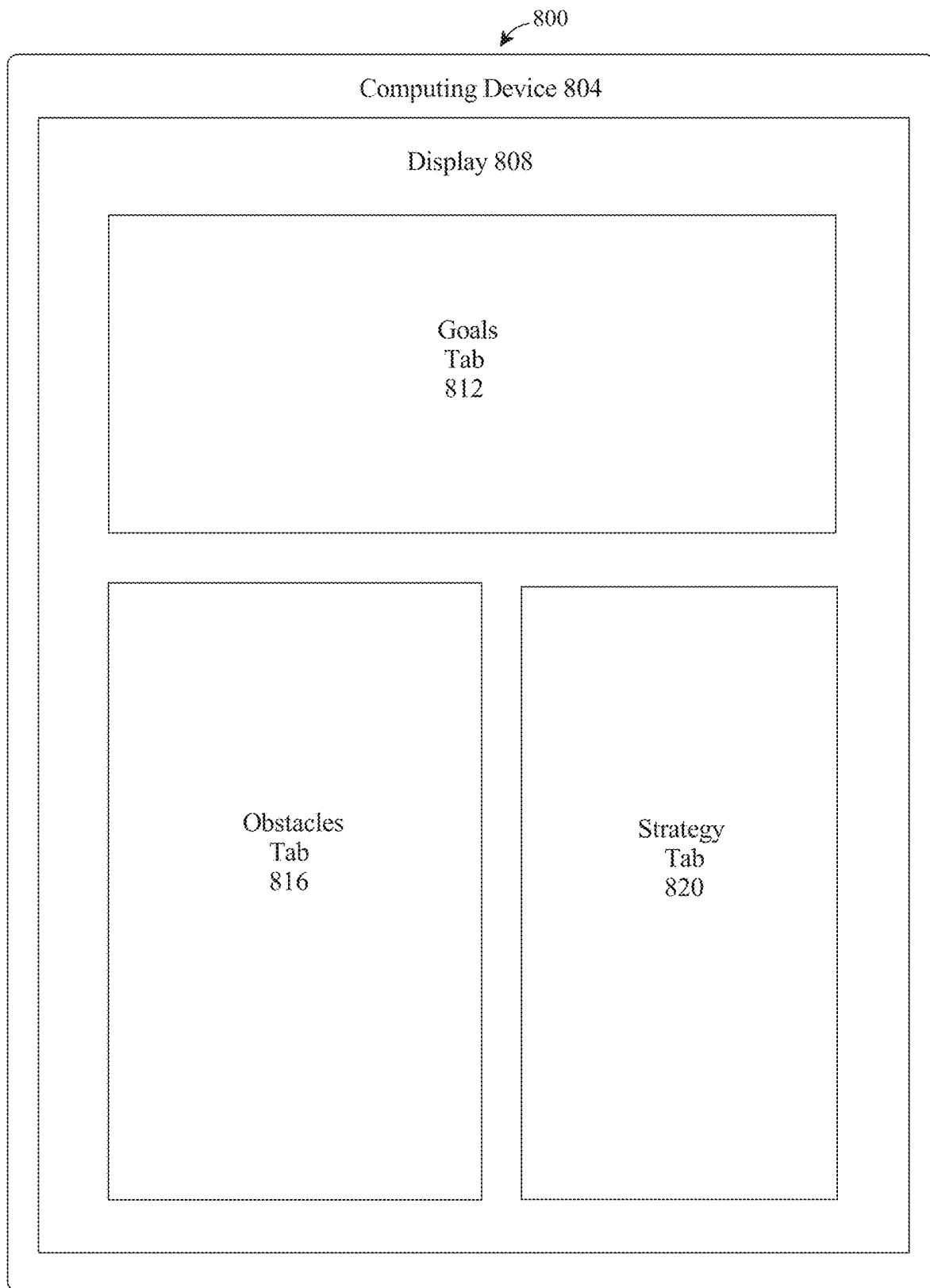


FIG. 6

**FIG. 7**

**FIG. 8**

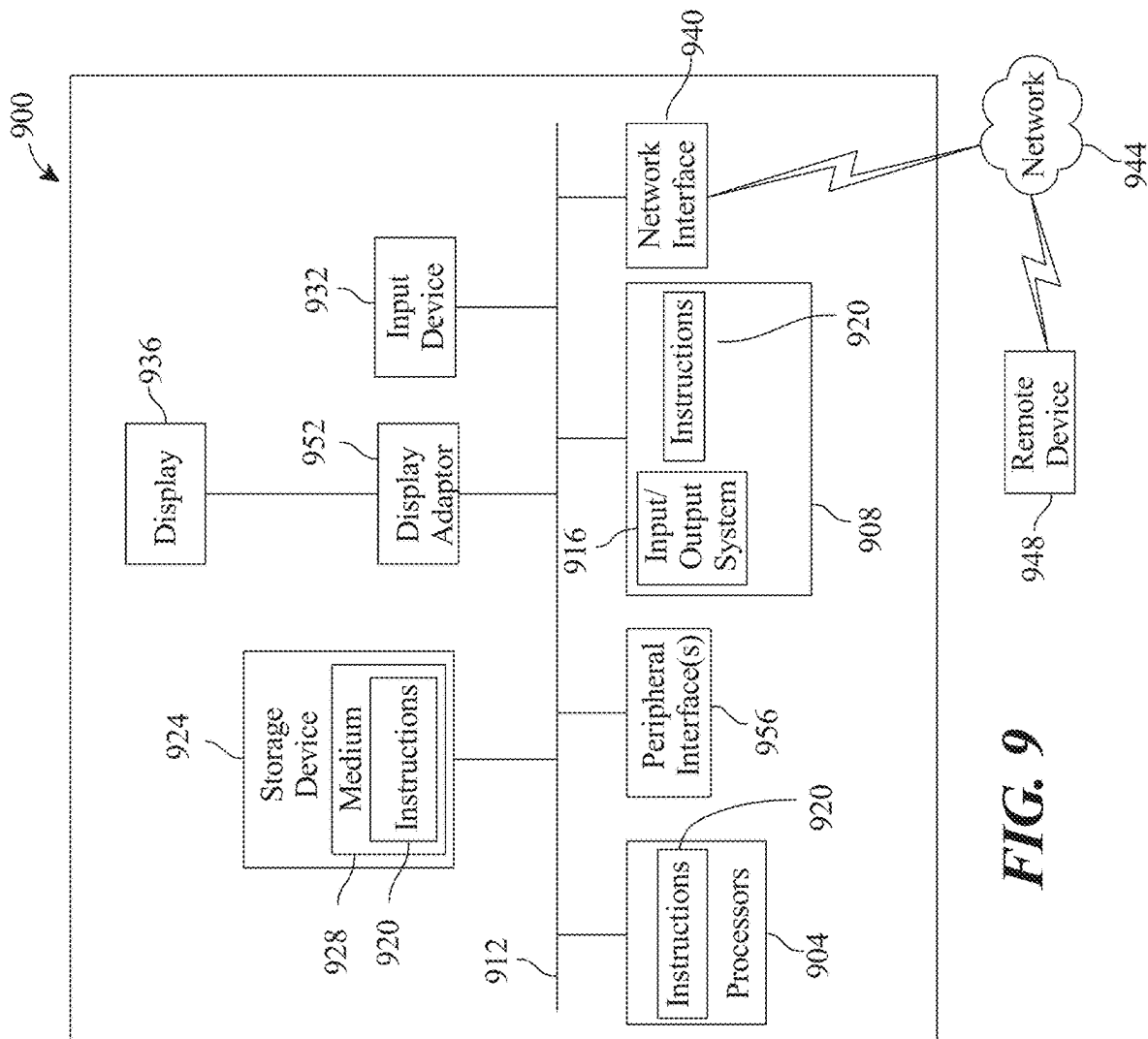


FIG. 9

1

**APPARATUS AND METHOD FOR DIRECTED
PROCESS GENERATION****CROSS-REFERENCE TO RELATED
APPLICATIONS**

This application is a continuation of Non-provisional application Ser. No. 18/142,656 filed on May 3, 2023, and entitled "APPARATUS AND METHOD FOR DIRECTED PROCESS GENERATION," which is incorporated herein by reference in its entirety.

FIELD OF THE INVENTION

The present invention generally relates to the field of machine learning. In particular, the present invention is directed to an apparatus and method for strategic goal development.

BACKGROUND

Automated process generation is difficult to perform accurately and efficiently in currently existent systems. A lack of defined constraints and over-variability of data has made conventional methods untenable.

SUMMARY OF THE DISCLOSURE

In an aspect an apparatus for directed process generation includes at least a processor and a memory communicatively connected to the at least a processor, wherein the memory containing instructions configuring the at least a processor to communicate, utilizing a user interface, an inquiry to a user, receive, using the user interface, a user profile from a user, determine at least a user goal as a function of the user profile, receive conditions data from a conditions database, identify, for each of the plurality of user goals, at least an obstacle datum as a function of the at least a user goal and conditions data, rank the plurality of user goals as a function of a quantity of the one or more obstacle datum associated with each of the plurality of user goals, generate a directed process as a function of the at least an obstacle datum and the at least a user goal, and display the directed process at a user device.

In another aspect, a method for directed process generation includes communicating, by at least a processor and utilizing a user interface, an inquiry to a user, receiving, by the at least a processor and using the user interface, a user profile from a user, determining, by the at least a processor, at least a user goal as a function of the user profile, receiving, by the at least a processor, conditions data from a conditions database, identifying, by the at least a processor and for each of the plurality of user goals, at least an obstacle datum as a function of the at least a user goal and conditions data, ranking, by the at least a processor, the plurality of user goals as a function of a quantity of the one or more obstacle datum associated with each of the plurality of user goals, generating, by the at least a processor, a directed process as a function of the at least an obstacle datum and the at least a user goal, and displaying, by the at least a processor, the directed process at a user device.

These and other aspects and features of non-limiting embodiments of the present invention will become apparent to those skilled in the art upon review of the following description of specific non-limiting embodiments of the invention in conjunction with the accompanying drawings.

2

BRIEF DESCRIPTION OF THE DRAWINGS

For the purpose of illustrating the invention, the drawings show aspects of one or more embodiments of the invention. However, it should be understood that the present invention is not limited to the precise arrangements and instrumentalities shown in the drawings, wherein:

FIG. 1 is a block diagram of an exemplary embodiment of an apparatus for strategic goal development;

FIG. 2 illustrates an exemplary embodiment of a chatbot;

FIG. 3 is a block diagram of an exemplary embodiment of a machine learning process;

FIG. 4 is a diagram of an exemplary embodiment of a neural network;

FIG. 5 is a diagram of an illustrative embodiment of a node of a neural network;

FIG. 6 is a graph illustrating an exemplary relationship between fuzzy sets;

FIG. 7 is a flow diagram of an exemplary method for strategic goal development;

FIG. 8 is an exemplary embodiment of a user interface;

FIG. 9 is a block diagram of a computing system that can be used to implement any one or more of the methodologies disclosed herein and any one or more portions thereof.

The drawings are not necessarily to scale and may be illustrated by phantom lines, diagrammatic representations and fragmentary views. In certain instances, details that are not necessary for an understanding of the embodiments or that render other details difficult to perceive may have been omitted.

DETAILED DESCRIPTION

At a high level, aspects of the present disclosure are directed to an apparatus and method for strategic goal development. In an embodiment, goals for a user may be identified using a machine learning model.

Aspects of the present disclosure allow for generating strategies for a user based on machine learning. Exemplary embodiments illustrating aspects of the present disclosure are described below in the context of several specific examples.

Referring now to FIG. 1, an exemplary embodiment of an apparatus **100** for directed process generation is illustrated. Apparatus **100** includes a processor **104**. Processor **104** may include any computing device as described in this disclosure, including without limitation a microcontroller, micro-processor, digital signal processor (DSP) and/or system on a chip (SoC) as described in this disclosure. Processor **104** may include, be included in, and/or communicate with a mobile device such as a mobile telephone or smartphone. Processor **104** may include a single computing device operating independently, or may include two or more computing device operating in concert, in parallel, sequentially or the like; two or more computing devices may be included together in a single computing device or in two or more computing devices. Processor **104** may interface or communicate with one or more additional devices as described below in further detail via a network interface device. Network interface device may be utilized for connecting processor **104** to one or more of a variety of networks, and one or more devices. Examples of a network interface device include, but are not limited to, a network interface card (e.g., a mobile network interface card, a LAN card), a modem, and any combination thereof. Examples of a network include, but are not limited to, a wide area network (e.g., the Internet, an enterprise network), a local area network (e.g., a network

associated with an office, a building, a campus or other relatively small geographic space), a telephone network, a data network associated with a telephone/voice provider (e.g., a mobile communications provider data and/or voice network), a direct connection between two computing devices, and any combinations thereof. A network may employ a wired and/or a wireless mode of communication. In general, any network topology may be used. Information (e.g., data, software etc.) may be communicated to and/or from a computer and/or a computing device. Processor **104** may include but is not limited to, for example, a computing device or cluster of computing devices in a first location and a second computing device or cluster of computing devices in a second location. Processor **104** may include one or more computing devices dedicated to data storage, security, distribution of traffic for load balancing, and the like. Processor **104** may distribute one or more computing tasks as described below across a plurality of computing devices of computing device, which may operate in parallel, in series, redundantly, or in any other manner used for distribution of tasks or memory between computing devices. Processor **104** may be implemented using a “shared nothing” architecture in which data is cached at the worker, in an embodiment, this may enable scalability of apparatus **100** and/or processor **104**.

With continued reference to FIG. 1, processor **104** may be designed and/or configured to perform any method, method step, or sequence of method steps in any embodiment described in this disclosure, in any order and with any degree of repetition. For instance, processor **104** may be configured to perform a single step or sequence repeatedly until a desired or commanded outcome is achieved; repetition of a step or a sequence of steps may be performed iteratively and/or recursively using outputs of previous repetitions as inputs to subsequent repetitions, aggregating inputs and/or outputs of repetitions to produce an aggregate result, reduction or decrement of one or more variables such as global variables, and/or division of a larger processing task into a set of iteratively addressed smaller processing tasks. Processor **104** may perform any step or sequence of steps as described in this disclosure in parallel, such as simultaneously and/or substantially simultaneously performing a step two or more times using two or more parallel threads, processor cores, or the like; division of tasks between parallel threads and/or processes may be performed according to any protocol suitable for division of tasks between iterations. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various ways in which steps, sequences of steps, processing tasks, and/or data may be subdivided, shared, or otherwise dealt with using iteration, recursion, and/or parallel processing.

Continuing to refer to FIG. 1, apparatus **100** includes a memory **108** communicatively connected to the at least a processor **104**, wherein the memory **108** contains instructions configuring processor **104** to perform tasks in accordance with this disclosure. As used in this disclosure, “communicatively connected” means connected by way of a connection, attachment or linkage between two or more relata which allows for reception and/or transmittance of information therebetween. For example, and without limitation, this connection may be wired or wireless, direct or indirect, and between two or more components, circuits, devices, systems, and the like, which allows for reception and/or transmittance of data and/or signal(s) therebetween. Data and/or signals therebetween may include, without limitation, electrical, electromagnetic, magnetic, video, audio, radio and microwave data and/or signals, combina-

tions thereof, and the like, among others. A communicative connection may be achieved, for example and without limitation, through wired or wireless electronic, digital or analog, communication, either directly or by way of one or more intervening devices or components. Further, communicative connection may include electrically coupling or connecting at least an output of one device, component, or circuit to at least an input of another device, component, or circuit. For example, and without limitation, via a bus or other facility for intercommunication between elements of a computing device. Communicative connecting may also include indirect connections via, for example and without limitation, wireless connection, radio communication, low power wide area network, optical communication, magnetic, capacitive, or optical coupling, and the like. In some instances, the terminology “communicatively coupled” may be used in place of communicatively connected in this disclosure.

Continuing to refer to FIG. 1, in some embodiments, memory **108** contains instructions configuring processor **104** to receive a user profile **112** from a profile database **116**. A “user profile,” as used herein, is a at least an element of data describing the user in some aspect. In a nonlimiting example, user profile **112** may be free-form text generated by a user that includes some descriptions of a user’s habits and tasks performed on a day-to-day basis. A “free-form text,” as used herein, are texts generated by a user without any formal constraints. An example, without limitations, is a short paragraph written by a user that describes the user in his or her own words. A “profile database,” as used herein, is a database configured to store user profile. Profile database **116** may be implemented, without limitation, as a relational database, a key-value retrieval database such as a NOSQL database, or any other format or structure for use as a database that a person skilled in the art would recognize as suitable upon review of the entirety of this disclosure. Database may alternatively or additionally be implemented using a distributed data storage protocol and/or data structure, such as a distributed hash table or the like. Database may include a plurality of data entries and/or records as described above. Data entries in a database may be flagged with or linked to one or more additional elements of information, which may be reflected in data entry cells and/or in linked tables such as tables related by one or more indices in a relational database. In some embodiments, user profile may be received from a user. In some embodiments, user profile may be received as a function of a user filling out a form. In further embodiments, form may include an online form. In other embodiments, form may be a physical form. In an embodiment, processor may be further configured to extract text from a physical form. In an embodiment, extracting text may include using an optical character reader (OCR). In a nonlimiting example, a user may fill out a form with information about themselves, such as using a pen to fill out a paper form, and processor **104** may extract text from the physical form using OCR. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various ways in which data entries in a database may store, retrieve, organize, and/or reflect data and/or records as used herein, as well as categories and/or populations of data consistently with this disclosure.

In some embodiments, processor **104** may determine user profile **112** using OCR. In some embodiments, optical character recognition or optical character reader (OCR) includes automatic conversion of images of written (e.g., typed, handwritten or printed text) into machine-encoded text. In some cases, recognition of at least a keyword from

5

an image component, such as an image provided with descriptions about where a user would like to travel in the future, may include one or more processes, including without limitation optical character recognition (OCR), optical word recognition, intelligent character recognition, intelligent word recognition, and the like. In some cases, OCR may recognize written text, one glyph or character at a time. In some cases, optical word recognition may recognize written text, one word at a time, for example, for languages that use a space as a word divider. In some cases, intelligent character recognition (ICR) may recognize written text one glyph or character at a time, for instance by employing machine learning processes. In some cases, intelligent word recognition (IWR) may recognize written text, one word at a time, for instance by employing machine learning processes.

Still referring to FIG. 1, in some cases OCR may be an “offline” process, which analyses a static document or image frame. In some cases, handwriting movement analysis can be used as input to handwriting recognition. For example, instead of merely using shapes of glyphs and words, this technique may capture motions, such as the order in which segments are drawn, the direction, and the pattern of putting the pen down and lifting it. This additional information can make handwriting recognition more accurate. In some cases, this technology may be referred to as “online” character recognition, dynamic character recognition, real-time character recognition, and intelligent character recognition.

Still referring to FIG. 1, in some cases, OCR processes may employ pre-processing of image component. Pre-processing process may include without limitation de-skew, de-speckle, binarization, line removal, layout analysis or “zoning,” line and word detection, script recognition, character isolation or “segmentation,” and normalization. In some cases, a de-skew process may include applying a transform (e.g., homography or affine transform) to image component to align text. In some cases, a de-speckle process may include removing positive and negative spots and/or smoothing edges. In some cases, a binarization process may include converting an image from color or greyscale to black-and-white (i.e., a binary image). Binarization may be performed as a simple way of separating text (or any other desired image component) from a background of image component. In some cases, binarization may be required for example if an employed OCR algorithm only works on binary images. In some cases, a line removal process may include removal of non-glyph or non-character imagery (e.g., boxes and lines). In some cases, a layout analysis or “zoning” process may identify columns, paragraphs, captions, and the like as distinct blocks. In some cases, a line and word detection process may establish a baseline for word and character shapes and separate words, if necessary. In some cases, a script recognition process may, for example in multilingual documents, identify script allowing an appropriate OCR algorithm to be selected. In some cases, a character isolation or “segmentation” process may separate signal characters, for example character-based OCR algorithms. In some cases, a normalization process may normalize aspect ratio and/or scale of image component.

Still referring to FIG. 1, in some embodiments an OCR process will include an OCR algorithm. Exemplary OCR algorithms include matrix matching process and/or feature extraction processes. Matrix matching may involve comparing an image to a stored glyph on a pixel-by-pixel basis. In some case, matrix matching may also be known as “pattern matching,” “pattern recognition,” and/or “image correlation.” Matrix matching may rely on an input glyph being

6

correctly isolated from the rest of the image component. Matrix matching may also rely on a stored glyph being in a similar font and at a same scale as input glyph. Matrix matching may work best with typewritten text.

Still referring to FIG. 1, in some embodiments, an OCR process may include a feature extraction process. In some cases, feature extraction may decompose a glyph into features. Exemplary non-limiting features may include corners, edges, lines, closed loops, line direction, line intersections, and the like. In some cases, feature extraction may reduce dimensionality of representation and may make the recognition process computationally more efficient. In some cases, extracted feature can be compared with an abstract vector-like representation of a character, which might reduce to one or more glyph prototypes. General techniques of feature detection in computer vision are applicable to this type of OCR. In some embodiments, machine-learning process like nearest neighbor classifiers (e.g., k-nearest neighbors algorithm) can be used to compare image features with stored glyph features and choose a nearest match. OCR may employ any machine-learning process described in this disclosure, for example machine-learning processes described with reference to FIGS. 3-6. Exemplary non-limiting OCR software includes Cuneiform and Tesseract. Cuneiform is a multi-language, open-source optical character recognition system originally developed by Cognitive Technologies of Moscow, Russia. Tesseract is free OCR software originally developed by Hewlett-Packard of Palo Alto, California, United States.

Still referring to FIG. 1, in some cases, OCR may employ a two-pass approach to character recognition. Second pass may include adaptive recognition and use letter shapes recognized with high confidence on a first pass to recognize better remaining letters on the second pass. In some cases, two-pass approach may be advantageous for unusual fonts or low-quality image components where visual verbal content may be distorted. Another exemplary OCR software tool include OCRopus. OCRopus development is led by German Research Centre for Artificial Intelligence in Kaiserslautern, Germany. In some cases, OCR software may employ neural networks, for example neural networks as taught in reference to FIGS. 3-6.

Still referring to FIG. 1, in some cases, OCR may include post-processing. For example, OCR accuracy can be increased, in some cases, if output is constrained by a lexicon. A lexicon may include a list or set of words that are allowed to occur in a document. In some cases, a lexicon may include, for instance, all the words in the English language, or a more technical lexicon for a specific field. In some cases, an output stream may be a plain text stream or file of characters. In some cases, an OCR process may preserve an original layout of visual verbal content. In some cases, near-neighbor analysis can make use of co-occurrence frequencies to correct errors, by noting that certain words are often seen together. For example, “Washington, D.C.” is generally far more common in English than “Washington DOC.” In some cases, an OCR process may make use of a priori knowledge of grammar for a language being recognized. For example, grammar rules may be used to help determine if a word is likely to be a verb or a noun. Distance conceptualization may be employed for recognition and classification. For example, a Levenshtein distance algorithm may be used in OCR post-processing to further optimize results.

With continued reference to FIG. 1, in an embodiment, processor 104 may be configured to receive user profile 112 as a function of an automatic speech recognition. In a

nonlimiting example, a user may dictate to a computing device the contents of user profile 112, such as answering some preset questions related to the user's habits and future plans. In some embodiments, automatic speech recognition may require training (i.e., enrollment). In some cases, training an automatic speech recognition model may require an individual speaker to read text or isolated vocabulary. In some cases, a solicitation video may include an audio component having an audible verbal content, the contents of which are known a priori by processor 104. Processor 104 may then train an automatic speech recognition model according to training data which includes audible verbal content correlated to known content. In this way, processor 104 may analyze a person's specific voice and train an automatic speech recognition model to the person's speech, resulting in increased accuracy. Processor 104 may be communicatively connected to a computing device. Alternatively or additionally, in some cases, computing device may include an automatic speech recognition model that is speaker-independent. As used in this disclosure, a "speaker independent" automatic speech recognition process does not require training for each individual speaker. Conversely, as used in this disclosure, automatic speech recognition processes that employ individual speaker specific training are "speaker dependent."

Still referring to FIG. 1, in some embodiments, an automatic speech recognition process may perform voice recognition or speaker identification. As used in this disclosure, "voice recognition" refers to identifying a speaker, from audio content, rather than what the speaker is saying. In some cases, processor 104 may first recognize a speaker of verbal audio content and then automatically recognize speech of the speaker, for example by way of a speaker dependent automatic speech recognition model or process. In some embodiments, an automatic speech recognition process can be used to authenticate or verify an identity of a speaker. In some cases, a speaker may or may not include subject. For example, subject may speak within solicitation video, but others may speak as well.

Still referring to FIG. 1, in some embodiments, an automatic speech recognition process may include one or all of acoustic modeling, language modeling, and statistically-based speech recognition algorithms. In some cases, an automatic speech recognition process may employ hidden Markov models (HMMs). As discussed in greater detail below, language modeling such as that employed in natural language processing applications like document classification or statistical machine translation, may also be employed by an automatic speech recognition process.

Still referring to FIG. 1, an exemplary algorithm employed in automatic speech recognition may include or even be based upon hidden Markov models. Hidden Markov models (HMMs) may include statistical models that output a sequence of symbols or quantities. HMMs can be used in speech recognition because a speech signal can be viewed as a piecewise stationary signal or a short-time stationary signal. For example, over a short time scale (e.g., 10 milliseconds), speech can be approximated as a stationary process. Speech (i.e., audible verbal content) can be understood as a Markov model for many stochastic purposes.

Still referring to FIG. 1, in some embodiments HMMs can be trained automatically and may be relatively simple and computationally feasible to use. In an exemplary automatic speech recognition process, a hidden Markov model may output a sequence of n-dimensional real-valued vectors (with n being a small integer, such as 10), at a rate of about one vector every 10 milliseconds. Vectors may consist of

cepstral coefficients. A cepstral coefficient requires using a spectral domain. Cepstral coefficients may be obtained by taking a Fourier transform of a short time window of speech yielding a spectrum, decorrelating the spectrum using a cosine transform, and taking first (i.e., most significant) coefficients. In some cases, an HMM may have in each state a statistical distribution that is a mixture of diagonal covariance Gaussians, yielding a likelihood for each observed vector. In some cases, each word, or phoneme, may have a different output distribution; an HMM for a sequence of words or phonemes may be made by concatenating an HMMs for separate words and phonemes.

Still referring to FIG. 1, in some embodiments, an automatic speech recognition process may use various combinations of a number of techniques in order to improve results. In some cases, a large-vocabulary automatic speech recognition process may include context dependency for phonemes. For example, in some cases, phonemes with different left and right context may have different realizations as HMM states. In some cases, an automatic speech recognition process may use cepstral normalization to normalize for different speakers and recording conditions. In some cases, an automatic speech recognition process may use vocal tract length normalization (VTLN) for male-female normalization and maximum likelihood linear regression (MLLR) for more general speaker adaptation. In some cases, an automatic speech recognition process may determine so-called delta and delta-delta coefficients to capture speech dynamics and might use heteroscedastic linear discriminant analysis (HLDA). In some cases, an automatic speech recognition process may use splicing and a linear discriminate analysis (LDA)-based projection, which may include heteroscedastic linear discriminant analysis or a global semi-tied covariance transform (also known as maximum likelihood linear transform [MLLT]). In some cases, an automatic speech recognition process may use discriminative training techniques, which may dispense with a purely statistical approach to HMM parameter estimation and instead optimize some classification-related measure of training data; examples may include maximum mutual information (MMI), minimum classification error (MCE), and minimum phone error (MPE).

Still referring to FIG. 1, in some embodiments, an automatic speech recognition process may be said to decode speech (i.e., audible verbal content). Decoding of speech may occur when an automatic speech recognition system is presented with a new utterance and must compute a most likely sentence. In some cases, speech decoding may include a Viterbi algorithm. A Viterbi algorithm may include a dynamic programming algorithm for obtaining a maximum a posteriori probability estimate of a most likely sequence of hidden states (i.e., Viterbi path) that results in a sequence of observed events. Viterbi algorithms may be employed in context of Markov information sources and hidden Markov models. A Viterbi algorithm may be used to find a best path, for example using a dynamically created combination hidden Markov model, having both acoustic and language model information, using a statically created combination hidden Markov model (e.g., finite state transducer [FST] approach).

Still referring to FIG. 1, in some embodiments, speech (i.e., audible verbal content) decoding may include considering a set of good candidates and not only a best candidate, when presented with a new utterance. In some cases, a better scoring function (i.e., re-scoring) may be used to rate each of a set of good candidates, allowing selection of a best candidate according to this refined score. In some cases, a set

of candidates can be kept either as a list (i.e., N-best list approach) or as a subset of models (i.e., a lattice). In some cases, re-scoring may be performed by optimizing Bayes risk (or an approximation thereof). In some cases, re-scoring may include optimizing for sentence (including keywords) that minimizes an expectancy of a given loss function with regards to all possible transcriptions. For example, re-scoring may allow selection of a sentence that minimizes an average distance to other possible sentences weighted by their estimated probability. In some cases, an employed loss function may include Levenshtein distance, although different distance calculations may be performed, for instance for specific tasks. In some cases, a set of candidates may be pruned to maintain tractability.

Still referring to FIG. 1, in some embodiments, an automatic speech recognition process may employ dynamic time warping (DTW)-based approaches. Dynamic time warping may include algorithms for measuring similarity between two sequences, which may vary in time or speed. For instance, similarities in walking patterns would be detected, even if in one video the person was walking slowly and if in another he or she were walking more quickly, or even if there were accelerations and deceleration during the course of one observation. DTW has been applied to video, audio, and graphics—indeed, any data that can be turned into a linear representation can be analyzed with DTW. In some cases, DTW may be used by an automatic speech recognition process to cope with different speaking (i.e., audible verbal content) speeds. In some cases, DTW may allow processor 104 to find an optimal match between two given sequences (e.g., time series) with certain restrictions. That is, in some cases, sequences can be “warped” non-linearly to match each other. In some cases, a DTW-based sequence alignment method may be used in context of hidden Markov models.

Still referring to FIG. 1, in some embodiments, an automatic speech recognition process may include a neural network. Neural network may include any neural network, for example those disclosed with reference to FIGS. 3-6. In some cases, neural networks may be used for automatic speech recognition, including phoneme classification, phoneme classification through multi-objective evolutionary algorithms, isolated word recognition, audiovisual speech recognition, audiovisual speaker recognition and speaker adaptation. In some cases, neural networks employed in automatic speech recognition may make fewer explicit assumptions about feature statistical properties than HMMs and therefore may have several qualities making them attractive recognition models for speech recognition. When used to estimate the probabilities of a speech feature segment, neural networks may allow discriminative training in a natural and efficient manner. In some cases, neural networks may be used to effectively classify audible verbal content over short-time interval, for instance such as individual phonemes and isolated words. In some embodiments, a neural network may be employed by automatic speech recognition processes for pre-processing, feature transformation and/or dimensionality reduction, for example prior to HMM-based recognition. In some embodiments, long short-term memory (LSTM) and related recurrent neural networks (RNNs) and Time Delay Neural Networks (TDNN's) may be used for automatic speech recognition, for example over longer time intervals for continuous speech recognition.

Continuing to refer to FIG. 1, in an embodiment, user profile may be received as a function of an interaction between a user and a chatbot. A “user,” as used herein, is a person, or entity, that utilizes apparatus 100. As used in the current disclosure, a “chatbot” is a computer program

designed to simulate conversation with a user. In a nonlimiting example, chatbot may prompt user to provide information about their aspirations in life, such as a question about where a user sees himself/herself in 5 years, to which a user may provide information, in voice or text form, which will be used to determine at least a goal for the user. Chatbot is described in more detail in FIG. 2.

Still referring to FIG. 1, in an embodiment, processor 104 is configured to identify at least a user goal 120 using user profile 112. A “goal,” as used herein, is a task or an objective that the user would like to achieve. In embodiments, a user goal 120 may be a pecuniary goal, improving pecuniary literacy, musical goal, a vocational goal, an educational goal, a skill goal, a health goal, fitness goal, and the like. As used in the current disclosure, a “vocational goal” is a goal that is related to improving a user's career. As used in the current disclosure, a “pecuniary goal” is a goal that is related to improving a user's pecuniary position. As used in the current disclosure, a “pecuniary literacy” is a user's knowledge of how to improve his or her pecuniary position. As used in the current disclosure, an “educational goal” is an element of data used to improve the users formal or professional education. Examples of goals may include saving money, improving pecuniary literacy, buying a home, establishing an emergency fund, optimizing the family budget, starting a new career field, improving overall health, fitness goals, and the like. In a further non-limiting example, processor 104 may generate an educational goal for the user of getting a college degree. In another non-limiting example, processor 104 may generate a pecuniary goal for the user to save a predetermined amount of money. In some embodiments, processor 104 may be configured to generate a goals list, wherein the goals list includes a plurality of user goals 120. In some embodiments, processor 104 may be further configured to rank each goal from goals list. In a nonlimiting example, user goals 120 may be ranked based on feasibility of accomplishing those goals, such as how many steps each goal would require to be accomplished. Ranking may include any ranking method described in this disclosure. It will be apparent to one of ordinary skill in the art, upon reading this disclosure, of the many goals that can be identified for a user and the many ways that those goals can be ranked.

Continuing to refer to FIG. 1, in an embodiment, processor 104 is further configured to receive conditions data 124 from a conditions database 128. As used herein, “conditions data” are conditions that may affect accomplishing a goal. In a nonlimiting example, conditions data 124 may include financial data that affects a vocational goal, such as wage growth in a specific sector. In another nonlimiting example, conditions data 124 may include increases in price index of items related to pecuniary goals, such as increases in prices for commonly bought grocery items. Conditions database 128 may include any database described throughout this disclosure. Persons skilled in the art, upon reading this disclosure, will appreciate that conditions data 124 are described as way of example and may include a plurality of conditions not described in this disclosure.

With continued reference to FIG. 1, in embodiments, processor 104 is further configured to identify at least an obstacle datum 132 as a function of the user goal 120 and the conditions data 124. An “obstacle datum,” as used herein, is an element of data describing conditions that negatively impact achievement of a user goal. In a nonlimiting example, obstacle datum 132 may include a heart condition of the user that may affect a fitness goal. In another non-limiting example, obstacle datum 132 may include a require-

11

ment for pursuing an advanced academic degree that may affect a user's educational goal.

Still referring to FIG. 1, in embodiments, processor 104 is further configured to generate a directed process 136 as a function of the obstacle datum 132 and the user goal 120. A “directed process,” as used herein, are a set of steps required to achieve a user goal 120. In an embodiment, directed process 136 may include a plurality of instructions regarding how to achieve user goal 120. In some embodiments, directed process 136 may be a user goal 120 broken down into a series of sub-goals. In some embodiments, the sub-goals may be smaller or more simple goals used to progress the user towards user goal 120. In a nonlimiting example, if processor 104 generates a pecuniary user goal 120 of purchasing a car, directed process 136 may include the steps of 1. identifying a user's credit, which may be a condition of low score; 2. identifying a price range; 3. promoting the users to increase credit score, which may include the sub-step of paying-off credit card debt; 4. identify a vehicle within the price range of the user; 5. get financing on the purchase based on credit score; and 6. Generate an offer for purchase of the car. Additionally, directed process 136 may include combination of steps and sub steps. A step may include a task that a user must complete in order to achieve user goal 120. Once a user has achieved a plurality of steps and sub-steps the user may achieve a waypoint. A “waypoint,” as used herein, is at least a step required for performing all the steps in directed process 136. In embodiments, directed process 136 may include one or more waypoints. A further non-limiting example, a waypoint may be to increase credit rating to a certain score as to allow user to finance a purchase. Continuing to refer to FIG. 1, in some embodiments, processor 104 is further configured to transmit the directed process 136 to a user device 140.

With continued reference to FIG. 1, in some embodiments, identifying the at least an obstacle datum 132 may further comprise training an obstacle classifier 144 using obstacle training data 148. As used herein, an “obstacle classifier,” is a machine learning model configured to label a condition as obstacle datum 132 based on correlations of conditions to goals. In embodiments, obstacle training data 148 may include prior outputs of obstacle classifier 144. In embodiments, obstacle training data may include exemplary correlations of conditions to goals. In an embodiment, obstacle training data 148 may be included in conditions database 128. In embodiments, obstacle classifier 144 and obstacle training data 148 may include any algorithm and/or method described in reference to FIGS. 3-6.

Continuing to refer to FIG. 1, in embodiments, identifying the at least an obstacle datum 132 may further comprise classifying at least a condition datum 152 from the conditions data 124 to the at least a user goal 120 using the obstacle classifier 144.

Still referring to FIG. 1, in some embodiments, identifying the at least an obstacle datum 132 may further comprise generating the at least an obstacle datum 132 as a function of the classification.

With continued reference to FIG. 1, in embodiments, processor 104 may be further configured to determine the at least a user goal 120 as a function of a goal setting machine learning model 156. In some embodiments, goal setting machine learning model 156 may be trained using a goals training set 160. As used herein, a “goal setting machine learning model” is a data structure storing and/or implementing mathematical and/or algorithmic representation of a relationship between inputs and outputs. Inputs to goal setting machine learning model 156 may include, as non-

12

limiting examples, financial statements, physical attributes, such as Body Mass Index (BMI) number, employment data, education level and the like. Outputs, as nonlimiting examples, may include goals such as improving finances, reducing weight, change of employment and the like. Goals training set 160 may include exemplary inputs correlated to exemplary outputs, such mock data. In embodiments, goals training set 160 may include prior iterations of goal setting machine learning model 156. Goals training set 160, in embodiments, may include prior user goals 120 generated for the user. In some embodiments, goals training set 160 may include prior user goals 120 generated for other users. In embodiments, goals training dataset 160 may include data input by a user, such as through a form. In some embodiments, goals training dataset 160 may be labelled by a user. In some embodiments, goals training dataset 160 may include data input by a user through user device 140. Goal setting machine learning model 156 and goals training set 160 may be consistent with any machine learning model and training data described in reference to FIGS. 2-6.

Continuing to refer to FIG. 1, in an embodiment, processor 104 may be configured to generate a modified goal 164. In some embodiments, processor 104 may be configured to generate modified goal 164 as a function of goal setting machine learning model 156. A “modified goal,” as used herein, is a goal similar to user goal 120 that may be more easily achievable. In some embodiments, processor 104 may be configured to identify at least an obstacle datum 132 as a function of the modified goal 164. In embodiments, processor 104 may be configured to generate modified goal 164 when a user goal 120 cannot be achieved. In some embodiments, processor 104 may be configured to generate modified goal 120 when a step in directed process 136 cannot be achieved. In a nonlimiting example, a user's credit score may be so low that financing for a user goal 120 of purchasing that a vehicle within a higher price range cannot be achieved. In this nonlimiting example, processor 104 may generate a modified goal 164 of purchasing a vehicle within a lower price range.

Now referring to FIG. 2, an illustrative embodiment of a chatbot system 200 is presented. According to some embodiments, a user interface 204 may be communicative with a computing device 208 that is configured to operate a chatbot. In some cases, user interface 204 may be local to computing device 208. Alternatively or additionally, in some cases, user interface 204 may remote to computing device 208 and communicative with the computing device 208, by way of one or more networks, such as without limitation the internet. In some embodiments, computing device 208 may include user device 140. In embodiments, computing device 208 may be communicatively connected to processor 104. Alternatively or additionally, user interface 204 may communicate with computing device 208 using telephonic devices and networks, such as without limitation fax machines, short message service (SMS), or multimedia message service (MMS). Commonly, user interface 204 communicates with computing device 208 using text-based communication, for example without limitation using a character encoding protocol, such as American Standard for Information Interchange (ASCII). Typically, a user interface 204 conversationally interfaces a chatbot, by way of at least a submission 212, from the user interface 204 to the chatbot, and a response 216, from the chatbot to the user interface 204. In many cases, one or both of submission 212 and response 216 are text-based communication. Alternatively or additionally, in some cases, one or both of submission 212 and response 216 are audio-based communication.

13

Continuing in reference to FIG. 2, a submission 212 once received by computing device 208 operating a chatbot, may be processed by a processor 220. In some embodiments, processor 220 processes a submission 212 using one or more of keyword recognition, pattern matching, and natural language processing. In some embodiments, processor employs real-time learning with evolutionary algorithms. In some cases, processor 220 may retrieve a pre-prepared response from at least a storage component 224, based upon submission 212. Alternatively or additionally, in some embodiments, processor 220 communicates a response 216 without first receiving a submission 212, thereby initiating conversation. In some cases, processor 220 communicates an inquiry to user interface 204; and the processor is configured to process an answer to the inquiry in a following submission 212 from the user interface 204. In a nonlimiting example, an answer to an inquiry present within a submission 212 from user device 140 may be used by processor 104 as an input to another function, for example without limitation user profile 106.

Referring now to FIG. 3, an exemplary embodiment of a machine-learning module 300 that may perform one or more machine-learning processes as described in this disclosure is illustrated. Machine-learning module may perform determinations, classification, and/or analysis steps, methods, processes, or the like as described in this disclosure using machine learning processes. A “machine learning process,” as used in this disclosure, is a process that automatically uses training data 304 to generate an algorithm that will be performed by a computing device/module to produce outputs 308 given data provided as inputs 312; this is in contrast to a non-machine learning software program where the commands to be executed are determined in advance by a user and written in a programming language.

Still referring to FIG. 3, “training data,” as used herein, is data containing correlations that a machine-learning process may use to model relationships between two or more categories of data elements. For instance, and without limitation, training data 304 may include a plurality of data entries, each entry representing a set of data elements that were recorded, received, and/or generated together; data elements may be correlated by shared existence in a given data entry, by proximity in a given data entry, or the like. Multiple data entries in training data 304 may evince one or more trends in correlations between categories of data elements; for instance, and without limitation, a higher value of a first data element belonging to a first category of data element may tend to correlate to a higher value of a second data element belonging to a second category of data element, indicating a possible proportional or other mathematical relationship linking values belonging to the two categories. Multiple categories of data elements may be related in training data 304 according to various correlations; correlations may indicate causative and/or predictive links between categories of data elements, which may be modeled as relationships such as mathematical relationships by machine-learning processes as described in further detail below. Training data 304 may be formatted and/or organized by categories of data elements, for instance by associating data elements with one or more descriptors corresponding to categories of data elements. As a non-limiting example, training data 304 may include data entered in standardized forms by persons or processes, such that entry of a given data element in a given field in a form may be mapped to one or more descriptors of categories. Elements in training data 304 may be linked to descriptors of categories by tags, tokens, or other data elements; for instance, and without limitation, training data

14

304 may be provided in fixed-length formats, formats linking positions of data to categories such as comma-separated value (CSV) formats and/or self-describing formats such as extensible markup language (XML), JavaScript Object Notation (JSON), or the like, enabling processes or devices to detect categories of data.

Alternatively or additionally, and continuing to refer to FIG. 3, training data 304 may include one or more elements that are not categorized; that is, training data 304 may not be formatted or contain descriptors for some elements of data. Machine-learning algorithms and/or other processes may sort training data 304 according to one or more categorizations using, for instance, natural language processing algorithms, tokenization, detection of correlated values in raw data and the like; categories may be generated using correlation and/or other processing algorithms. As a non-limiting example, in a corpus of text, phrases making up a number “n” of compound words, such as nouns modified by other nouns, may be identified according to a statistically significant prevalence of n-grams containing such words in a particular order; such an n-gram may be categorized as an element of language such as a “word” to be tracked similarly to single words, generating a new category as a result of statistical analysis. Similarly, in a data entry including some textual data, a person’s name may be identified by reference to a list, dictionary, or other compendium of terms, permitting ad-hoc categorization by machine-learning algorithms, and/or automated association of data in the data entry with descriptors or into a given format. The ability to categorize data entries automatically may enable the same training data 304 to be made applicable for two or more distinct machine-learning algorithms as described in further detail below. Training data 304 used by machine-learning module 300 may correlate any input data as described in this disclosure to any output data as described in this disclosure. As a non-limiting illustrative example, need for transportation may be an input and a closely associated goal, such as financing a vehicle, may be an output.

Further referring to FIG. 3, training data may be filtered, sorted, and/or selected using one or more supervised and/or unsupervised machine-learning processes and/or models as described in further detail below; such models may include without limitation a training data classifier 316. Training data classifier 316 may include a “classifier,” which as used in this disclosure is a machine-learning model as defined below, such as a mathematical model, neural net, or program generated by a machine learning algorithm known as a “classification algorithm,” as described in further detail below, that sorts inputs into categories or bins of data, outputting the categories or bins of data and/or labels associated therewith. A classifier may be configured to output at least a datum that labels or otherwise identifies a set of data that are clustered together, found to be close under a distance metric as described below, or the like. A distance metric may include any norm, such as, without limitation, a Pythagorean norm. Machine-learning module 300 may generate a classifier using a classification algorithm, defined as a processes whereby a computing device and/or any module and/or component operating thereon derives a classifier from training data 304. Classification may be performed using, without limitation, linear classifiers such as without limitation logistic regression and/or naive Bayes classifiers, nearest neighbor classifiers such as k-nearest neighbors classifiers, support vector machines, least squares support vector machines, fisher’s linear discriminant, quadratic classifiers, decision trees, boosted trees,

15

random forest classifiers, learning vector quantization, and/or neural network-based classifiers.

Still referring to FIG. 3, machine-learning module 300 may be configured to perform a lazy-learning process 320 and/or protocol, which may alternatively be referred to as a “lazy loading” or “call-when-needed” process and/or protocol, may be a process whereby machine learning is conducted upon receipt of an input to be converted to an output, by combining the input and training set to derive the algorithm to be used to produce the output on demand. For instance, an initial set of simulations may be performed to cover an initial heuristic and/or “first guess” at an output and/or relationship. As a non-limiting example, an initial heuristic may include a ranking of associations between inputs and elements of training data 304. Heuristic may include selecting some number of highest-ranking associations and/or training data 304 elements. Lazy learning may implement any suitable lazy learning algorithm, including without limitation a K-nearest neighbors algorithm, a lazy naïve Bayes algorithm, or the like; persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various lazy-learning algorithms that may be applied to generate outputs as described in this disclosure, including without limitation lazy learning applications of machine-learning algorithms as described in further detail below.

Alternatively or additionally, and with continued reference to FIG. 3, machine-learning processes as described in this disclosure may be used to generate machine-learning models 324. A “machine-learning model,” as used in this disclosure, is a mathematical and/or algorithmic representation of a relationship between inputs and outputs, as generated using any machine-learning process including without limitation any process as described above and stored in memory; an input is submitted to a machine-learning model 324 once created, which generates an output based on the relationship that was derived. For instance, and without limitation, a linear regression model, generated using a linear regression algorithm, may compute a linear combination of input data using coefficients derived during machine-learning processes to calculate an output datum. As a further non-limiting example, a machine-learning model 324 may be generated by creating an artificial neural network, such as a convolutional neural network comprising an input layer of nodes, one or more intermediate layers, and an output layer of nodes. Connections between nodes may be created via the process of “training” the network, in which elements from a training data 304 set are applied to the input nodes, a suitable training algorithm (such as Levenberg-Marquardt, conjugate gradient, simulated annealing, or other algorithms) is then used to adjust the connections and weights between nodes in adjacent layers of the neural network to produce the desired values at the output nodes. This process is sometimes referred to as deep learning.

Still referring to FIG. 3, machine-learning algorithms may include at least a supervised machine-learning process 328. At least a supervised machine-learning process 328, as defined herein, include algorithms that receive a training set relating a number of inputs to a number of outputs, and seek to find one or more mathematical relations relating inputs to outputs, where each of the one or more mathematical relations is optimal according to some criterion specified to the algorithm using some scoring function. For instance, a supervised learning algorithm may include inputs as described in this disclosure as inputs, outputs as described in this disclosure as outputs, and a scoring function representing a desired form of relationship to be detected between inputs and outputs; scoring function may, for instance, seek

16

to maximize the probability that a given input and/or combination of elements inputs is associated with a given output to minimize the probability that a given input is not associated with a given output. Scoring function may be expressed as a risk function representing an “expected loss” of an algorithm relating inputs to outputs, where loss is computed as an error function representing a degree to which a prediction generated by the relation is incorrect when compared to a given input-output pair provided in training data 304. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various possible variations of at least a supervised machine-learning process 328 that may be used to determine relation between inputs and outputs. Supervised machine-learning processes may include classification algorithms as defined above.

Further referring to FIG. 3, machine learning processes may include at least an unsupervised machine-learning processes 332. An unsupervised machine-learning process, as used herein, is a process that derives inferences in datasets without regard to labels; as a result, an unsupervised machine-learning process may be free to discover any structure, relationship, and/or correlation provided in the data. Unsupervised processes may not require a response variable; unsupervised processes may be used to find interesting patterns and/or inferences between variables, to determine a degree of correlation between two or more variables, or the like.

Still referring to FIG. 3, machine-learning module 300 may be designed and configured to create a machine-learning model 324 using techniques for development of linear regression models. Linear regression models may include ordinary least squares regression, which aims to minimize the square of the difference between predicted outcomes and actual outcomes according to an appropriate norm for measuring such a difference (e.g. a vector-space distance norm); coefficients of the resulting linear equation may be modified to improve minimization. Linear regression models may include ridge regression methods, where the function to be minimized includes the least-squares function plus term multiplying the square of each coefficient by a scalar amount to penalize large coefficients. Linear regression models may include least absolute shrinkage and selection operator (LASSO) models, in which ridge regression is combined with multiplying the least-squares term by a factor of 1 divided by double the number of samples. Linear regression models may include a multi-task lasso model wherein the norm applied in the least-squares term of the lasso model is the Frobenius norm amounting to the square root of the sum of squares of all terms. Linear regression models may include the elastic net model, a multi-task elastic net model, a least angle regression model, a LARS lasso model, an orthogonal matching pursuit model, a Bayesian regression model, a logistic regression model, a stochastic gradient descent model, a perceptron model, a passive aggressive algorithm, a robustness regression model, a Huber regression model, or any other suitable model that may occur to persons skilled in the art upon reviewing the entirety of this disclosure. Linear regression models may be generalized in an embodiment to polynomial regression models, whereby a polynomial equation (e.g. a quadratic, cubic or higher-order equation) providing a best predicted output/actual output fit is sought; similar methods to those described above may be applied to minimize error functions, as will be apparent to persons skilled in the art upon reviewing the entirety of this disclosure.

Continuing to refer to FIG. 3, machine-learning algorithms may include, without limitation, linear discriminant

17

analysis. Machine-learning algorithm may include quadratic discriminant analysis. Machine-learning algorithms may include kernel ridge regression. Machine-learning algorithms may include support vector machines, including without limitation support vector classification-based regression processes. Machine-learning algorithms may include stochastic gradient descent algorithms, including classification and regression algorithms based on stochastic gradient descent. Machine-learning algorithms may include nearest neighbors algorithms. Machine-learning algorithms may include various forms of latent space regularization such as variational regularization. Machine-learning algorithms may include Gaussian processes such as Gaussian Process Regression. Machine-learning algorithms may include cross-decomposition algorithms, including partial least squares and/or canonical correlation analysis. Machine-learning algorithms may include naïve Bayes methods. Machine-learning algorithms may include algorithms based on decision trees, such as decision tree classification or regression algorithms. Machine-learning algorithms may include ensemble methods such as bagging meta-estimator, forest of randomized trees, AdaBoost, gradient tree boosting, and/or voting classifier methods. Machine-learning algorithms may include neural net algorithms, including convolutional neural net processes.

Referring now to FIG. 4, an exemplary embodiment of neural network **400** is illustrated. A neural network **400** also known as an artificial neural network, is a network of “nodes,” or data structures having one or more inputs, one or more outputs, and a function determining outputs based on inputs. Such nodes may be organized in a network, such as without limitation a convolutional neural network, including an input layer of nodes **404**, one or more intermediate layers **408**, and an output layer of nodes **412**. Connections between nodes may be created via the process of “training” the network, in which elements from a training dataset are applied to the input nodes, a suitable training algorithm (such as Levenberg-Marquardt, conjugate gradient, simulated annealing, or other algorithms) is then used to adjust the connections and weights between nodes in adjacent layers of the neural network to produce the desired values at the output nodes. This process is sometimes referred to as deep learning. Connections may run solely from input nodes toward output nodes in a “feed-forward” network or may feed outputs of one layer back to inputs of the same or a different layer in a “recurrent network.” As a further non-limiting example, a neural network may include a convolutional neural network comprising an input layer of nodes, one or more intermediate layers, and an output layer of nodes. A “convolutional neural network,” as used in this disclosure, is a neural network in which at least one hidden layer is a convolutional layer that convolves inputs to that layer with a subset of inputs known as a “kernel,” along with one or more additional layers such as pooling layers, fully connected layers, and the like.

Referring now to FIG. 5, an exemplary embodiment of a node **500** of a neural network is illustrated. A node may include, without limitation, a plurality of inputs x , that may receive numerical values from inputs to a neural network containing the node and/or from other nodes. Node may perform one or more activation functions to produce its output given one or more inputs, such as without limitation computing a binary step function comparing an input to a threshold value and outputting either a logic 1 or logic 0 output or something equivalent, a linear activation function whereby an output is directly proportional to the input, and/or a non-linear activation function, wherein the output is

18

not proportional to the input. Non-linear activation functions may include, without limitation, a sigmoid function of the form

$$f(x) = \frac{1}{1 - e^{-x}}$$

given input x , a tanh (hyperbolic tangent) function, of the form

$$\frac{e^x - e^{-x}}{e^x + e^{-x}},$$

a tanh derivative function such as $f(x) = \tanh^2(x)$, a rectified linear unit function such as $f(x) = \max(0, x)$, a “leaky” and/or “parametric” rectified linear unit function such as $f(x) = \max(ax, x)$ for some a , an exponential linear units function such as

$$f(x) = \begin{cases} x & \text{for } x \geq 0 \\ \alpha(e^x - 1) & \text{for } x < 0 \end{cases}$$

for some value of α (this function may be replaced and/or weighted by its own derivative in some embodiments), a softmax function such as

$$f(x_i) = \frac{e^{x_i}}{\sum_i x_i}$$

where the inputs to an instant layer are x_i , a swish function such as $f(x) = x * \text{sigmoid}(x)$, a Gaussian error linear unit function such as $f(x) = a(1 + \tanh(\sqrt{2/\pi}(x + bx^r)))$ for some values of a , b , and r , and/or a scaled exponential linear unit function such as

$$f(x) = \lambda \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}.$$

Fundamentally, there is no limit to the nature of functions of inputs x_i that may be used as activation functions. As a non-limiting and illustrative example, node may perform a weighted sum of inputs using weights w_i that are multiplied by respective inputs x_i . Additionally or alternatively, a bias b may be added to the weighted sum of the inputs such that an offset is added to each unit in the neural network layer that is independent of the input to the layer. The weighted sum may then be input into a function ϕ , which may generate one or more outputs y . Weight w_i applied to an input x_i may indicate whether the input is “excitatory,” indicating that it has strong influence on the one or more outputs y , for instance by the corresponding weight having a large numerical value, and/or a “inhibitory,” indicating it has a weak effect influence on the one more inputs y , for instance by the corresponding weight having a small numerical value. The values of weights w , may be determined by training a neural network using training data, which may be performed using any suitable process as described above.

Referring now to FIG. 6, an exemplary embodiment of fuzzy set comparison **600** is illustrated. A first fuzzy set **604** may be represented, without limitation, according to a first

membership function **608** representing a probability that an input falling on a first range of values **612** is a member of the first fuzzy set **604**, where the first membership function **608** has values on a range of probabilities such as without limitation the interval [0,1], and an area beneath the first membership function **608** may represent a set of values within first fuzzy set **604**. Although first range of values **612** is illustrated for clarity in this exemplary depiction as a range on a single number line or axis, first range of values **612** may be defined on two or more dimensions, representing, for instance, a Cartesian product between a plurality of ranges, curves, axes, spaces, dimensions, or the like. First membership function **608** may include any suitable function mapping first range **612** to a probability interval, including without limitation a triangular function defined by two linear elements such as line segments or planes that intersect at or below the top of the probability interval. As a non-limiting example, triangular membership function may be defined as:

$$y(x, a, b, c) = \begin{cases} 0, & \text{for } x > c \text{ and } x < a \\ \frac{x-a}{b-a}, & \text{for } a \leq x < b \\ \frac{c-x}{c-b}, & \text{if } b < x \leq c \end{cases}$$

a trapezoidal membership function may be defined as:

$$y(x, a, b, c, d) = \max\left(\min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right), 0\right)$$

a sigmoidal function may be defined as:

$$y(x, a, c) = \frac{1}{1 + e^{-a(x-c)}}$$

a Gaussian membership function may be defined as:

$$y(x, c, \sigma) = e^{-\frac{1}{2}\left(\frac{x-c}{\sigma}\right)^2}$$

and a bell membership function may be defined as:

$$y(x, a, b, c) = \left[1 + \left|\frac{x-c}{a}\right|^{2b}\right]^{-1}$$

Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various alternative or additional membership functions that may be used consistently with this disclosure.

Still referring to FIG. 6, first fuzzy set **604** may represent any value or combination of values as described above, including output from one or more machine-learning models. A second fuzzy set **616**, which may represent any value which may be represented by first fuzzy set **604**, may be defined by a second membership function **620** on a second range **624**; second range **624** may be identical and/or overlap with first range **612** and/or may be combined with first range via Cartesian product or the like to generate a mapping permitting evaluation overlap of first fuzzy set **604** and second fuzzy set **616**. Where first fuzzy set **604** and second fuzzy set **616** have a region **628** that overlaps, first membership function **608** and second membership function **620**

may intersect at a point **632** representing a probability, as defined on probability interval, of a match between first fuzzy set **604** and second fuzzy set **616**. Alternatively or additionally, a single value of first and/or second fuzzy set may be located at a locus **636** on first range **612** and/or second range **624**, where a probability of membership may be taken by evaluation of first membership function **608** and/or second membership function **620** at that range point. A probability at **628** and/or **632** may be compared to a threshold **640** to determine whether a positive match is indicated. Threshold **640** may, in a non-limiting example, represent a degree of match between first fuzzy set **604** and second fuzzy set **616**, and/or single values therein with each other or with either set, which is sufficient for purposes of the matching process; for instance, threshold may indicate a sufficient degree of overlap between an output from one or more machine-learning models. Alternatively or additionally, each threshold may be tuned by a machine-learning and/or statistical process, for instance and without limitation as described in further detail below.

Further referring to FIG. 6, in an embodiment, a degree of match between fuzzy sets may be used to classify a task with a current role **132**. For instance, if a task has a fuzzy set matching a task associated with a role fuzzy set by having a degree of overlap exceeding a threshold, processor **102** may classify the condition as belonging to a goal. Where multiple fuzzy matches are performed, degrees of match for each respective fuzzy set may be computed and aggregated through, for instance, addition, averaging, or the like, to determine an overall degree of match.

Still referring to FIG. 6, in an embodiment, a task may be compared to multiple user state fuzzy sets. For instance, task may be represented by a fuzzy set that is compared to each of the multiple current role fuzzy sets; and a degree of overlap exceeding a threshold between the condition fuzzy set and any of the multiple current role fuzzy sets may cause apparatus **100** to classify the condition as belonging to a goal. For instance, in one embodiment there may be two goals fuzzy sets, representing respectively a pecuniary and a health goal. Pecuniary goal may have a pecuniary fuzzy set; health goal may have a health fuzzy set; and condition may have a condition fuzzy set. Processor **104**, for example, may compare a condition fuzzy set with each of pecuniary fuzzy set and health fuzzy set, as described above, and classify a condition to either, both, or neither of pecuniary or health. Machine-learning methods as described throughout may, in a non-limiting example, generate coefficients used in fuzzy set equations as described above, such as without limitation x , c , and σ of a Gaussian set as described above, as outputs of machine-learning methods. Likewise, task may be used indirectly to determine a fuzzy set, as task fuzzy set may be derived from outputs of one or more machine-learning models that take the task directly or indirectly as inputs.

Now referring to FIG. 7, a method **700** for strategic goal development is presented. Method **700**, at step **705**, includes receiving, by processor **104**, user profile **112**. In an embodiment, method **700** further includes receiving user profile **112** using an optical character reader (OCR). In an embodiment, method **700** may include receiving user profile **112** as a function of an interaction between a user and a chatbot.

Continuing to refer to FIG. 7, method **700**, at step **710**, includes determining at least a user goal **120** as a function of user profile **112**. In an embodiment, method **700** may include determining at least a user goal **120** as a function of goal setting machine learning model **156**. In further embodiments. Method **700** may include generating modified goal

164 as function of goal setting machine learning model 156. In some embodiments, method 700 may include generating a goals list, wherein the goals list include a plurality of user goals 120. In further embodiments, method 700 may include ranking each goal from the goals list. In a non-limiting example, goals may be ranked based on feasibility, such as higher ranking for goals that have fewer obstacles associated with them.

Still referring to FIG. 7, at step 715, method 700 includes receiving conditions data 124 from conditions database 128. Conditions database 128 may include any database described throughout this disclosure.

With continued reference to FIG. 7, at step 720, method 700 includes identifying at least an obstacle datum 132 as a function of at least a user goal 120 and conditions data 124. In some embodiments, identifying at least an obstacle datum 132 may include training obstacle classifier 144 using obstacle training data 148, wherein obstacle training data 148 includes obstacles correlated to a plurality of goals. In further embodiments, identifying at least an obstacle datum 132 may include classifying at least a condition datum 152 to at least a user goal 120 using obstacle classifier 144. In further embodiments, identifying at least an obstacle datum 132 may include generating at least an obstacle datum 132 as function of the classification. In some embodiments, method 700 may include identifying at least an obstacle datum 132 as a function of modified goal 164.

Continuing to refer to FIG. 7, method 700, at step 725, includes generating directed process 136 as function of at least an obstacle 132 and at least a user goal 120. In some embodiments, method 700 may include generating directed process 136 as function of a goals ranking. In a nonlimiting example, directed process 136 may prioritize accomplishing goals with higher ranking, such as prioritizing a strategy for exercises related to improving cardiovascular health before recommending exercises related to gaining muscular mass.

Still referring to FIG. 7, method 700, at step 730, includes transmitting directed process 136 to user device 140. In some embodiments, user device 140 may include a chatbot.

Referring now to FIG. 8, an exemplary embodiment of a user interface 800 is presented. In embodiments, user interface 800 is configured to be displayed in a computing device 804. In embodiments, computing device 804 may include any device capable of displaying a Graphical User Interface (GUI). In an embodiment, user interface 800 may be a Graphical User Interface (GUI). In embodiments, computing device 804 may be configured to display any data structure described throughout this disclosure. In nonlimiting examples, computing device 804 may be a smartphone, smartwatch, laptop, desktop computer, virtual reality headset and the like. In embodiments, computing device 804 may include a display 808. Display 808 may include any display capable of presenting information in graphical form. In embodiments, Display 808 may be configured to display a GUI. In embodiments, computing device 804 may include a plurality of displays 808. In embodiments, display 808 is configured to display a window. A "window," as used herein, is a graphical representation that occupies at least a position of the display 808. In embodiments, window may occupy the entire display 808. In some embodiments, display 808 may be communicatively connected to computing device 804. In some embodiments, a window may include one or more tabs. A "tab," as used herein, is a subsection of a window. In a nonlimiting example, computing device 804 may transmit information to display 808. In an example, without limitations, a user may navigate through a first second, third and fourth window (and so on) by interacting

with display 808. In another nonlimiting example, a user may navigate through a first second, third and fourth window (and so on) by interacting directly with computing device 804, and indirectly with display 808, such as when information is being transmitted to a remote display 808. Display 808 may further contain event handlers, wherein the placement of text within a textbox may signify to computing device 804, or display 808, to display another window. An "event handler" as used in this disclosure is a callback routine that operates asynchronously once an event takes place. Event handlers may include, without limitation, one or more programs to perform one or more actions based on user input, such as generating pop-up windows, submitting forms, requesting more information, and the like. For example, an event handler may be programmed to request more information or may be programmed to generate messages following a user input. User input may include clicking buttons, mouse clicks, hovering of a mouse, input using a touchscreen, keyboard clicks, an entry of characters, entry of symbols, an upload of an image, an upload of a computer file, manipulation of computer icons, and the like. For example, an event handler may be programmed to generate a notification screen following a user input wherein the notification screen notifies a user that the data was properly received. In some embodiments, an event handler may be programmed to request additional information after a first user input is received. In some embodiments, an event handler may be programmed to generate a pop-up notification when a user input is left blank. In some embodiments, an event handler may be programmed to generate requests based on the user input.

Continuing to refer to FIG. 8, in an embodiment, computing device 804 may be configured to display a goals tab 812. A "goals tab," as used herein, is a subsection of a window configured to display at least a user goal 120. In embodiments, computing device 804 may be configured to display an obstacles tab 816. An "obstacle tab," as used herein is a subsection of a window configured to display at least an obstacle datum 132. In embodiments, computing device 804 may be configured to display a strategy tab 820. A "strategy tab," as used herein, is a subsection of a window configured to display directed process 136. It will be apparent to one of ordinary skill in the art, upon reading this disclosure, that user interface 800 is described as a way of example only, and that user interface 800 may include a plurality of configurations not described herein.

It is to be noted that any one or more of the aspects and embodiments described herein may be conveniently implemented using one or more machines (e.g., one or more computing devices that are utilized as a user computing device for an electronic document, one or more server devices, such as a document server, etc.) programmed according to the teachings of the present specification, as will be apparent to those of ordinary skill in the computer art. Appropriate software coding can readily be prepared by skilled programmers based on the teachings of the present disclosure, as will be apparent to those of ordinary skill in the software art. Aspects and implementations discussed above employing software and/or software modules may also include appropriate hardware for assisting in the implementation of the machine executable instructions of the software and/or software module.

Such software may be a computer program product that employs a machine-readable storage medium. A machine-readable storage medium may be any medium that is capable of storing and/or encoding a sequence of instructions for execution by a machine (e.g., a computing device) and that

causes the machine to perform any one of the methodologies and/or embodiments described herein. Examples of a machine-readable storage medium include, but are not limited to, a magnetic disk (e.g., CD, CD-R, DVD, DVD-R, etc.), a magneto-optical disk, a read-only memory “ROM” device, a random access memory “RAM” device, a magnetic card, an optical card, a solid-state memory device, an EPROM, an EEPROM, and any combinations thereof. A machine-readable medium, as used herein, is intended to include a single medium as well as a collection of physically separate media, such as, for example, a collection of compact discs or one or more hard disk drives in combination with a computer memory. As used herein, a machine-readable storage medium does not include transitory forms of signal transmission.

Such software may also include information (e.g., data) carried as a data signal on a data carrier, such as a carrier wave. For example, machine-executable information may be included as a data-carrying signal embodied in a data carrier in which the signal encodes a sequence of instruction, or portion thereof, for execution by a machine (e.g., a computing device) and any related information (e.g., data structures and data) that causes the machine to perform any one of the methodologies and/or embodiments described herein.

Examples of a computing device include, but are not limited to, an electronic book reading device, a computer workstation, a terminal computer, a server computer, a handheld device (e.g., a tablet computer, a smartphone, etc.), a web appliance, a network router, a network switch, a network bridge, any machine capable of executing a sequence of instructions that specify an action to be taken by that machine, and any combinations thereof. In one example, a computing device may include and/or be included in a kiosk.

FIG. 9 shows a diagrammatic representation of one embodiment of a computing device in the exemplary form of a computer system 900 within which a set of instructions for causing a control system to perform any one or more of the aspects and/or methodologies of the present disclosure may be executed. It is also contemplated that multiple computing devices may be utilized to implement a specially configured set of instructions for causing one or more of the devices to perform any one or more of the aspects and/or methodologies of the present disclosure. Computer system 900 includes a processor 904 and a memory 908 that communicate with each other, and with other components, via a bus 912. Bus 912 may include any of several types of bus structures including, but not limited to, a memory bus, a memory controller, a peripheral bus, a local bus, and any combinations thereof, using any of a variety of bus architectures.

Processor 904 may include any suitable processor, such as without limitation a processor incorporating logical circuitry for performing arithmetic and logical operations, such as an arithmetic and logic unit (ALU), which may be regulated with a state machine and directed by operational inputs from memory and/or sensors; processor 904 may be organized according to Von Neumann and/or Harvard architecture as a non-limiting example. Processor 904 may include, incorporate, and/or be incorporated in, without limitation, a microcontroller, microprocessor, digital signal processor (DSP), Field Programmable Gate Array (FPGA), Complex Programmable Logic Device (CPLD), Graphical Processing Unit (GPU), general purpose GPU, Tensor Processing Unit (TPU), analog or mixed signal processor, Trusted Platform Module (TPM), a floating point unit (FPU), and/or system on a chip (SoC).

Memory 908 may include various components (e.g., machine-readable media) including, but not limited to, a random-access memory component, a read only component, and any combinations thereof. In one example, a basic input/output system 916 (BIOS), including basic routines that help to transfer information between elements within computer system 900, such as during start-up, may be stored in memory 908. Memory 908 may also include (e.g., stored on one or more machine-readable media) instructions (e.g., software) 920 embodying any one or more of the aspects and/or methodologies of the present disclosure. In another example, memory 908 may further include any number of program modules including, but not limited to, an operating system, one or more application programs, other program modules, program data, and any combinations thereof.

Computer system 900 may also include a storage device 924. Examples of a storage device (e.g., storage device 924) include, but are not limited to, a hard disk drive, a magnetic disk drive, an optical disc drive in combination with an optical medium, a solid-state memory device, and any combinations thereof. Storage device 924 may be connected to bus 912 by an appropriate interface (not shown). Example interfaces include, but are not limited to, SCSI, advanced technology attachment (ATA), serial ATA, universal serial bus (USB), IEEE 1394 (FIREWIRE), and any combinations thereof. In one example, storage device 924 (or one or more components thereof) may be removably interfaced with computer system 900 (e.g., via an external port connector (not shown)). Particularly, storage device 924 and an associated machine-readable medium 928 may provide nonvolatile and/or volatile storage of machine-readable instructions, data structures, program modules, and/or other data for computer system 900. In one example, software 920 may reside, completely or partially, within machine-readable medium 928. In another example, software 920 may reside, completely or partially, within processor 904.

Computer system 900 may also include an input device 932. In one example, a user of computer system 900 may enter commands and/or other information into computer system 900 via input device 932. Examples of an input device 932 include, but are not limited to, an alpha-numeric input device (e.g., a keyboard), a pointing device, a joystick, a gamepad, an audio input device (e.g., a microphone, a voice response system, etc.), a cursor control device (e.g., a mouse), a touchpad, an optical scanner, a video capture device (e.g., a still camera, a video camera), a touchscreen, and any combinations thereof. Input device 932 may be interfaced to bus 912 via any of a variety of interfaces (not shown) including, but not limited to, a serial interface, a parallel interface, a game port, a USB interface, a FIREWIRE interface, a direct interface to bus 912, and any combinations thereof. Input device 932 may include a touch screen interface that may be a part of or separate from display 936, discussed further below. Input device 932 may be utilized as a user selection device for selecting one or more graphical representations in a graphical interface as described above.

A user may also input commands and/or other information to computer system 900 via storage device 924 (e.g., a removable disk drive, a flash drive, etc.) and/or network interface device 940. A network interface device, such as network interface device 940, may be utilized for connecting computer system 900 to one or more of a variety of networks, such as network 944, and one or more remote devices 948 connected thereto. Examples of a network interface device include, but are not limited to, a network interface card (e.g., a mobile network interface card, a LAN

25

card), a modem, and any combination thereof. Examples of a network include, but are not limited to, a wide area network (e.g., the Internet, an enterprise network), a local area network (e.g., a network associated with an office, a building, a campus or other relatively small geographic space), a telephone network, a data network associated with a telephone/voice provider (e.g., a mobile communications provider data and/or voice network), a direct connection between two computing devices, and any combinations thereof. A network, such as network 944, may employ a wired and/or a wireless mode of communication. In general, any network topology may be used. Information (e.g., data, software 920, etc.) may be communicated to and/or from computer system 900 via network interface device 940.

Computer system 900 may further include a video display adapter 952 for communicating a displayable image to a display device, such as display device 936. Examples of a display device include, but are not limited to, a liquid crystal display (LCD), a cathode ray tube (CRT), a plasma display, a light emitting diode (LED) display, and any combinations thereof. Display adapter 952 and display device 936 may be utilized in combination with processor 904 to provide graphical representations of aspects of the present disclosure. In addition to a display device, computer system 900 may include one or more other peripheral output devices including, but not limited to, an audio speaker, a printer, and any combinations thereof. Such peripheral output devices may be connected to bus 912 via a peripheral interface 956. Examples of a peripheral interface include, but are not limited to, a serial port, a USB connection, a FIREWIRE connection, a parallel connection, and any combinations thereof.

The foregoing has been a detailed description of illustrative embodiments of the invention. Various modifications and additions can be made without departing from the spirit and scope of this invention. Features of each of the various embodiments described above may be combined with features of other described embodiments as appropriate in order to provide a multiplicity of feature combinations in associated new embodiments. Furthermore, while the foregoing describes a number of separate embodiments, what has been described herein is merely illustrative of the application of the principles of the present invention. Additionally, although particular methods herein may be illustrated and/or described as being performed in a specific order, the ordering is highly variable within ordinary skill to achieve methods, systems, and software according to the present disclosure. Accordingly, this description is meant to be taken only by way of example, and not to otherwise limit the scope of this invention.

Exemplary embodiments have been disclosed above and illustrated in the accompanying drawings. It will be understood by those skilled in the art that various changes, omissions and additions may be made to that which is specifically disclosed herein without departing from the spirit and scope of the present invention.

What is claimed is:

1. An apparatus for directed process generation, wherein the apparatus comprises:

at least a processor; and
a memory communicatively connected to the at least a processor, wherein the memory contains instructions configuring the at least a processor to:
communicate, utilizing a user interface, an inquiry to a user;

26

receive, using the user interface, a user profile from a user, wherein the user profile comprises information related to a vocational goal of the user;

generate a training data classifier as a function of unfiltered training data using a classification algorithm;

filter elements of the unfiltered training data using the training data classifier to generate a plurality of training data sets each containing a plurality of data entries correlating a plurality of user profile data and a plurality of user goal categories;

select at least one filtered training data set of the plurality of training data sets as a function of the vocational goal of the user using the training data classifier;

train a machine-learning model as a function of the at least one filtered training data set;

determine a plurality of user goals as a function of the user profile using the trained machine-learning model;

receive conditions data from a conditions database;

identify, for each of the plurality of user goals, at least an obstacle datum as a function of the plurality of user goals and conditions data;

rank the plurality of user goals as a function of a quantity of each at least an obstacle datum associated with each of the plurality of user goals;

generate a directed process as a function of each at least an obstacle datum and each associated user goal; and
display the directed process at a user device.

2. The apparatus of claim 1, wherein the user interface includes a graphical user interface, the graphical user interface including a subsection for receiving user goal data.

3. The apparatus of claim 1, wherein the user interface further includes a chatbot, and the at least a processor is further configured to receive the user profile as a function of an interaction between the user and the chatbot.

4. The apparatus of claim 1, wherein the at least a processor is configured to receive the user profile using an optical character reader.

5. The apparatus of claim 1, wherein the at least a processor is configured to receive the use profile using automatic speech recognition.

6. The apparatus of claim 1, wherein the machine-learning model comprises a goal setting machine-learning model.

7. The apparatus of claim 6, wherein the at least a processor is further configured to generate a modified goal as a function of the goal setting machine-learning model.

8. The apparatus of claim 7, wherein the at least a processor is further configured to identify the at least an obstacle datum as a function of the modified goal.

9. The apparatus of claim 6, wherein the goal setting machine-learning model is a neural network.

10. The apparatus of claim 1, wherein the at least a processor is further configured to generate the directed process as a function of the ranking.

11. A method for directed process generation, wherein the method comprises:

communicating, by at least a processor and utilizing a user interface, an inquiry to a user;

receiving, by the at least a processor and using the user interface, a user profile from a user, wherein the user profile comprises information related to a vocational goal of the user;

generating, by the at least a processor, a training data classifier as a function of unfiltered training data using a classification algorithm;

27

filtering, by the at least a processor, elements of the unfiltered training data using the training data classifier to generate a plurality of training data sets each containing a plurality of data entries correlating a plurality of user profile data and a plurality of user goal categories; 5

selecting, by the at least a processor, at least one filtered training data set of the plurality of training data sets as a function of the vocational goal of the user using the training data classifier; 10

training, by the at least a processor, a machine-learning model as a function of the at least one filtered training data set;

determining, by the at least a processor, a plurality of user goals as a function of the user profile using the trained machine-learning model; 15

receiving, by the at least a processor, conditions data from a conditions database;

identifying, by the at least a processor and for each of the plurality of user goals, at least an obstacle datum as a function of the plurality of user goals and conditions data; 20

ranking, by the at least a processor, the plurality of user goals as a function of a quantity of each obstacle datum associated with each of the plurality of user goals; 25

generating, by the at least a processor, a directed process as a function of each at least an obstacle datum and each associated user goal; and

28

displaying, by the at least a processor, the directed process at a user device.

12. The method of claim 11, wherein the user interface includes a graphical user interface, the graphical user interface including a subsection for receiving user goal data.

13. The method of claim 11, wherein the user interface further includes a chatbot, and the at least a processor is further configured to receive the user profile as a function of an interaction between the user and the chatbot.

14. The method of claim 11, wherein the at least a processor is configured to receive the user profile using an optical character reader.

15. The method of claim 11, wherein the at least a processor is configured to receive the use profile using automatic speech recognition.

16. The method of claim 11, wherein the machine-learning model comprises a goal setting machine-learning model.

17. The method of claim 16, wherein the at least a processor is further configured to generate a modified goal as a function of the goal setting machine-learning model.

18. The method of claim 17, wherein the at least a processor is further configured to identify the at least an obstacle datum as a function of the modified goal.

19. The method of claim 16, wherein the goal setting machine-learning model is a neural network.

20. The method of claim 11, wherein the at least a processor is further configured to generate the directed process as a function of the ranking.

* * * * *